CODE4LIFE TECHNOLOGY

LƯU LẠI ĐỂ ÔN THI

CẨM NANG ATTT

AN TOÀN THÔNG TIN CHO SINH VIÊN • 94 TRANG

```python
import hashlib

a = hashlib.sha256(b"123456").hexdigest()
b = hashlib.sha256(b"123456").hexdigest()
print(a[:16], a == b)
```

Đã băm mật khẩu. Vì sao vẫn chưa an toàn?

Mật mã | Xác thực | Web | Pháp luật | Ôn thi

Code4Life Technology - Tào Việt Đức

Bản đồ cẩm nang

Đi từ nguyên lý, qua mật mã và phòng thủ, tới pháp luật và ôn thi.

1 TỔNG QUAN Trang 3

2 MẬT MÃ Trang 11

3 XÁC THỰC VÀ PHÂN QUYỀN Trang 29

4 AN TOÀN ỨNG DỤNG WEB Trang 38

5 AN TOÀN HỆ THỐNG Trang 51

6 AN TOÀN MẠNG Trang 57

7 CON NGƯỜI VÀ VẬN HÀNH Trang 62

8 PHÁP LUẬT VÀ QUẢN TRỊ Trang 69

9 THỰC HÀNH Trang 73

10 ÔN THI Trang 81

Dùng cẩm nang này thế nào?

Cẩm nang viết theo hướng phòng thủ: hiểu tấn công hoạt động ra sao để nhận ra và ngăn chặn.

Mục tiêu của bạn	Nên đọc
Học môn An toàn thông tin	**Phần 1 đến 3, 5, 6**
Thi mật mã	**Phần 2, 10**
Lập trình web an toàn	**Phần 3, 4**
Bảo vệ tài khoản cá nhân	**Phần 7, 9**
Hiểu pháp luật, tiêu chuẩn	**Phần 8**

QUY TẮC SỐ MỘT

Chỉ thực hành trên máy của bạn hoặc hệ thống được phép bằng văn bản. Xâm nhập hệ thống của người khác có thể bị xử lý theo pháp luật.

Nội dung được đối chiếu với học phần IT4015 của ĐH Bách khoa Hà Nội, OWASP, NIST, ISO/IEC 27001 và pháp luật Việt Nam. Mỗi trường có đề cương riêng, hãy so lại với chương trình của bạn.

Một khái niệm, nhiều tên gọi

Tiếng Anh	Các tên hay gặp
Confidentiality	Tính bí mật, tính bảo mật
Integrity	Tính toàn vẹn
Availability	Tính sẵn sàng, tính khả dụng
Authentication	Xác thực
Authorization	Phân quyền, cấp quyền
Non-repudiation	Chống chối bỏ
Encryption	Mã hóa
Hash	Hàm băm
Digital signature	Chữ ký số
Vulnerability	Lỗ hổng, điểm yếu
Threat	Mối đe dọa
Malware	Phần mềm độc hại, mã độc
Phishing	Lừa đảo trực tuyến, tấn công giả mạo
Incident response	Ứng cứu sự cố

An toàn thông tin, an ninh mạng và bảo mật thường được dùng gần nghĩa nhau. Văn bản pháp luật có định nghĩa riêng cho từng thuật ngữ.

1 An toàn thông tin là gì?

Là bảo vệ thông tin và hệ thống khỏi truy cập, thay đổi, gián đoạn trái phép. Ba mục tiêu cốt lõi gọi là CIA.

Mục tiêu	Ý nghĩa	Ví dụ khi mất
Bí mật	Chỉ người được phép mới đọc được	Lộ bảng điểm, lộ mật khẩu
Toàn vẹn	Dữ liệu không bị sửa trái phép	Sửa điểm, đổi số tiền
Sẵn sàng	Dùng được khi cần	Web đăng ký học phần sập

XÁC THỰC
Biết chắc người dùng hoặc thông điệp là thật.

CHỐNG CHỐI BỎ
Người đã thực hiện không thể phủ nhận, thường nhờ chữ ký số.

Bảo mật tuyệt đối không tồn tại. Mục tiêu thực tế là giảm rủi ro xuống mức chấp nhận được, với chi phí hợp lý.

2 Tài sản, đe dọa, lỗ hổng, rủi ro

Bốn khái niệm nền tảng, đề thi hay hỏi phân biệt.

Khái niệm	Ý nghĩa	Ví dụ
Tài sản	Thứ có giá trị cần bảo vệ	Cơ sở dữ liệu sinh viên
Lỗ hổng	Điểm yếu có thể bị lợi dụng	Form đăng nhập lỗi SQL
Mối đe dọa	Điều có thể gây hại	Kẻ tấn công, thiên tai, lỗi người
Tấn công	Hành động lợi dụng lỗ hổng	Lấy cắp dữ liệu
Rủi ro	Khả năng xảy ra kèm mức thiệt hại	Lộ dữ liệu, bị phạt
Biện pháp	Cách giảm rủi ro	Vá lỗi, sao lưu, đào tạo

TẤN CÔNG CHỦ ĐỘNG
Thay đổi dữ liệu hoặc hệ thống: giả mạo, sửa, DoS.

TẤN CÔNG THỤ ĐỘNG
Chỉ quan sát: nghe lén, phân tích lưu lượng.

3 Các nguyên tắc thiết kế an toàn

Nguyên tắc	Ý nghĩa
Đặc quyền tối thiểu	Chỉ cấp đúng quyền cần để làm việc
Phòng thủ nhiều lớp	Một lớp hỏng thì còn lớp khác
Mặc định an toàn	Không chắc thì từ chối
Phân tách nhiệm vụ	Việc quan trọng cần hai người
Thiết kế mở	An toàn không dựa vào giấu cách làm
Kiểm tra mọi truy cập	Không tin kết quả kiểm tra cũ
Thu nhỏ bề mặt tấn công	Tắt dịch vụ không dùng
Đơn giản	Càng phức tạp càng dễ có lỗi

Nguyên lý Kerckhoffs: hệ mật mã phải an toàn ngay cả khi kẻ tấn công biết mọi thứ, trừ khóa.

4 Kẻ tấn công và các bước tấn công

Nhóm	Động cơ
Tội phạm mạng	Tiền: tống tiền, lừa đảo, bán dữ liệu
Nhóm được nhà nước hậu thuẫn	Tình báo, phá hoại
Hacktivist	Ý thức hệ, gây chú ý
Người bên trong	Trả thù, tiền, hoặc do sơ suất
Người mới, tò mò	Thử kỹ năng, nhưng vẫn gây hậu quả thật

Giai đoạn	Kẻ tấn công làm gì	Bên phòng thủ làm gì
Do thám	Thu thập thông tin	Giảm thông tin công khai
Xâm nhập ban đầu	Lừa đảo, khai thác lỗ hổng	Đào tạo, vá lỗi, MFA
Duy trì	Cài cửa hậu	Giám sát thay đổi
Di chuyển ngang	Lan sang máy khác	Phân vùng mạng
Đạt mục tiêu	Lấy dữ liệu, mã hóa đòi tiền	DLP, sao lưu, ứng cứu

Bên phòng thủ chỉ cần chặn được một mắt xích là có thể phá cả chuỗi.

5 Mô hình hóa mối đe dọa

Trước khi code, hãy hỏi: mình đang làm gì, cái gì có thể sai, mình sẽ làm gì, đã đủ tốt chưa?

Loại	Vi phạm	Biện pháp
Spoofing	Xác thực	MFA, chữ ký
Tampering	Toàn vẹn	Băm, MAC, chữ ký
Repudiation	Chống chối bỏ	Nhật ký, chữ ký số
Information disclosure	Bí mật	Mã hóa, phân quyền
Denial of service	Sẵn sàng	Giới hạn, dự phòng
Elevation of privilege	Phân quyền	Đặc quyền tối thiểu

Ví dụ với web đăng ký học phần: ai có thể giả làm sinh viên khác, sửa lớp, xem điểm người khác, làm sập hệ thống?

6 Đo rủi ro

Định tính dùng ma trận khả năng và tác động. Định lượng tính ra tiền, công thức hay ra đề là ALE.

```python
def level(likelihood, impact):      # thang 1 đến 5
    score = likelihood * impact
    if score >= 15:
        return score, "Cao"
    return score, "Trung bình" if score >= 6 else "Thấp"

print(level(4, 5), level(2, 3), level(1, 2))

asset = 1_000_000_000           # giá trị tài sản (AV)
ef = 0.4                        # tỉ lệ thiệt hại mỗi lần (EF)
aro = 0.5                       # số lần mỗi năm (ARO)
sle = asset * ef                # thiệt hại một lần
print(f"SLE = {sle:,.0f}")
print(f"ALE = {sle * aro:,.0f}")
```

KẾT QUẢ KHI CHẠY

```
(20, 'Cao') (6, 'Trung bình') (2, 'Thấp')
SLE = 400,000,000
ALE = 200,000,000
```

Cách xử lý rủi ro	Ý nghĩa
Giảm thiểu	Thêm biện pháp bảo vệ
Chuyển giao	Mua bảo hiểm, thuê ngoài
Né tránh	Ngừng hoạt động gây rủi ro
Chấp nhận	Ghi nhận và theo dõi

Một biện pháp đáng làm khi chi phí mỗi năm nhỏ hơn mức ALE giảm được.

7 Mật mã cổ điển

Không còn dùng trong thực tế, nhưng là bài mở đầu quen thuộc và hay ra đề.

```python
def shift(c, k):
    return chr((ord(c) - 65 + k) % 26 + 65)

def caesar(text, k):
    return "".join(shift(c, k) for c in text)

def vigenere(text, key, sign=1):
    return "".join(shift(c, sign * (ord(key[i % len(key)]) - 65))
                   for i, c in enumerate(text))

print(caesar("ANTOAN", 3))
c = vigenere("ANTOANTHONGTIN", "KEY")
print(c, vigenere(c, "KEY", -1))
```

KẾT QUẢ KHI CHẠY

```
DQWRDQ
KRRYELDLMXKRSR ANTOANTHONGTIN
```

Kiểu	Ý tưởng	Điểm yếu
Caesar	Dịch mỗi chữ k vị trí	Chỉ có 25 khóa
Thay thế đơn	Bảng thay chữ cố định	Lộ tần suất chữ cái
Vigenère	Dịch theo từ khóa lặp lại	Khóa lặp tạo ra chu kỳ
Chuyển vị	Đổi vị trí, giữ nguyên chữ	Giữ tần suất chữ cái

8 Vì sao mật mã cổ điển bị phá?

Không gian khóa nhỏ thì thử hết. Ngôn ngữ có quy luật thì phân tích tần suất.

```python
from collections import Counter

def shift(c, k):
    return chr((ord(c) - 65 + k) % 26 + 65) if c.isalpha() else c

plain = "THE ENEMY KNOWS THE SYSTEM SO THE KEY MUST STAY SECRET"
cipher = "".join(shift(c, 7) for c in plain)
print(cipher[:27])

letters = [c for c in cipher if c.isalpha()]
letter = Counter(letters).most_common(1)[0][0]
k = (ord(letter) - ord("E")) % 26     # đoán chữ hay gặp nhất là E
print(letter, k)
print("".join(shift(c, -k) for c in cipher)[:27])
```

KẾT QUẢ KHI CHẠY

```
AOL LULTF RUVDZ AOL ZFZALT
L 7
THE ENEMY KNOWS THE SYSTEM
```

Mô hình tấn công	Kẻ tấn công có gì
Chỉ có bản mã	Một số bản mã
Biết bản rõ	Một số cặp bản rõ, bản mã
Chọn bản rõ	Được chọn bản rõ để mã hóa
Chọn bản mã	Được chọn bản mã để giải mã

9 XOR và One-Time Pad

Khóa ngẫu nhiên thật, dài bằng thông điệp, dùng đúng một lần thì an toàn tuyệt đối về lý thuyết. Dùng lại khóa thì hỏng.

```python
def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

m1, m2 = b"HELLO", b"WORLD"
key = bytes([7, 42, 99, 200, 13])

c1, c2 = xor(m1, key), xor(m2, key)
print(c1.hex(), xor(c1, key))
print(xor(c1, c2) == xor(m1, m2))     # khóa bị triệt tiêu
```

KẾT QUẢ KHI CHẠY

```
4f6f2f8442 b'HELLO'
True
```

- Vì a XOR a = 0, nên XOR hai lần với cùng khóa cho lại dữ liệu gốc.
- Dùng lại khóa: XOR hai bản mã sẽ khử mất khóa, chỉ còn hai bản rõ trộn với nhau.
- Khó dùng thực tế vì phải trao khóa dài bằng dữ liệu một cách an toàn.
- Mã dòng hiện đại mô phỏng ý tưởng này bằng một dòng khóa sinh từ khóa ngắn.

Sai lầm kinh điển: dùng lại nonce trong mã dòng hoặc AES-CTR, về bản chất là dùng lại khóa.

10 Mã hóa đối xứng hiện đại

Hai bên dùng chung một khóa. Tốc độ cao nên dùng để mã hóa dữ liệu thật.

Thuật toán	Loại	Tình trạng
AES	Khối 128 bit, khóa 128, 192, 256	Tiêu chuẩn, nên dùng
ChaCha20	Mã dòng, khóa 256 bit	Nên dùng
DES	Khối 64 bit, khóa 56 bit	Đã bị phá, không dùng
3DES	Áp DES ba lần	Lỗi thời, không dùng mới
RC4	Mã dòng	Đã bị phá, không dùng

MÃ KHỐI
Xử lý theo khối cố định, cần chế độ hoạt động.

MÃ DÒNG
Tạo dòng khóa rồi XOR với dữ liệu.

Hai tính chất của mã tốt: gây lẫn lộn (giấu quan hệ giữa khóa và bản mã) và khuếch tán (một bit đổi kéo theo nhiều bit đổi).

Vấn đề lớn nhất của mã đối xứng là làm sao trao khóa an toàn, và n người cần n(n − 1)/2 khóa.

11 Chế độ hoạt động của mã khối

Chọn sai chế độ thì AES cũng hỏng. ECB làm lộ khuôn mẫu của dữ liệu.

```
import hashlib

def block(key, data):                # hàm mô phỏng, không phải AES
    return hashlib.sha256(key + data).digest()[:4]

def ecb(key, blocks):
    return [block(key, b).hex() for b in blocks]

def chained(key, blocks, iv):    # mô phỏng ý tưởng CBC
    out, prev = [], iv
    for b in blocks:
        prev = block(key, bytes(x ^ y for x, y in zip(b, prev)))
        out.append(prev.hex())
    return out

msg = [b"AAAA", b"BBBB", b"AAAA"]
print(ecb(b"k", msg))            # khối 1 và 3 giống nhau
print(chained(b"k", msg, b"\x01\x02\x03\x04"))
```

KẾT QUẢ KHI CHẠY

```
['d54693c4', '03b02dbb', 'd54693c4']
['2036abd7', 'a4c4aec9', 'c8d3adf9']
```

Chế độ	Đặc điểm	Khuyên dùng
ECB	Khối giống nhau cho bản mã giống nhau	Không
CBC	Cần IV ngẫu nhiên, không tự xác thực	Dùng kèm MAC
CTR	Biến mã khối thành mã dòng	Dùng kèm MAC
GCM	Mã hóa có xác thực (AEAD), nonce không lặp	Nên dùng

12 Độ dài khóa và vét cạn

Mỗi bit thêm vào làm số khóa tăng gấp đôi.

```python
def years(bits, rate=1e12):          # 10¹² phép thử mỗi giây
    return 2 ** (bits - 1) / rate / 31_557_600

for bits in (40, 56, 64, 128, 256):
    print(f"{bits:>3} bit: {years(bits):.2e} năm")
```

KẾT QUẢ KHI CHẠY

```
40 bit: 1.74e-08 năm
 56 bit: 1.14e-03 năm
 64 bit: 2.92e-01 năm
128 bit: 5.39e+18 năm
256 bit: 1.83e+57 năm
```

Đối xứng	RSA tương đương	ECC tương đương
112 bit	2048 bit	224 bit
128 bit	3072 bit	256 bit
256 bit	15360 bit	512 bit

- ✓ Công thức dùng trung bình một nửa không gian khóa.
- ✓ Tốc độ giả định chỉ để so sánh, không phải năng lực của bên nào cụ thể.
- ✓ Thực tế ít ai đoán khóa, họ tấn công mật khẩu, người dùng và lỗi cài đặt.

Bảng độ mạnh tương đương theo khuyến nghị NIST SP 800-57.

13 Số học modulo

Nền toán của RSA và Diffie-Hellman. Bài thi hay yêu cầu tính nghịch đảo modulo bằng Euclid mở rộng.

```python
def egcd(a, b):                     # trả về g, x, y với ax + by = g
    if b == 0:
        return a, 1, 0
    g, x, y = egcd(b, a % b)
    return g, y, x - a // b * y

def inverse(a, m):
    g, x, _ = egcd(a, m)
    if g != 1:
        return None                 # không có nghịch đảo
    return x % m

print(egcd(240, 46))
print(inverse(17, 3120), pow(17, -1, 3120))
print(inverse(6, 9))
print(pow(7, 128, 13))              # bình phương và nhân
```

KẾT QUẢ KHI CHẠY

```
(2, -9, 47)
2753 2753
None
3
```

Khái niệm	Ý nghĩa
a ≡ b (mod n)	a và b chia n cùng số dư
φ(n)	Số số nhỏ hơn n và nguyên tố cùng nhau với n
φ(p · q)	(p − 1)(q − 1) với p, q nguyên tố khác nhau
Định lý Euler	aˆφ(n) ≡ 1 (mod n) nếu gcd(a, n) = 1

14 RSA

Hai khóa: công khai để mã hóa, bí mật để giải mã. An toàn dựa trên độ khó phân tích n thành thừa số nguyên tố.

```
p, q = 61, 53                    # thực tế là số rất lớn
n = p * q
phi = (p - 1) * (q - 1)
e = 17
d = pow(e, -1, phi)
print("n =", n, "phi =", phi, "d =", d)

m = 65
c = pow(m, e, n)                 # mã hóa bằng khóa công khai
print("Bản mã:", c)
print("Giải mã:", pow(c, d, n))
```

KẾT QUẢ KHI CHẠY

```
n = 3233 phi = 3120 d = 2753
Bản mã: 2790
Giải mã: 65
```

1 Tạo khóa

Chọn p, q. Tính n, φ(n). Chọn e, tính $d = e^{-1} \bmod \varphi(n)$.

2 Mã hóa và giải mã

$c = m^e \bmod n$, và $m = c^d \bmod n$.

KÍCH THƯỚC

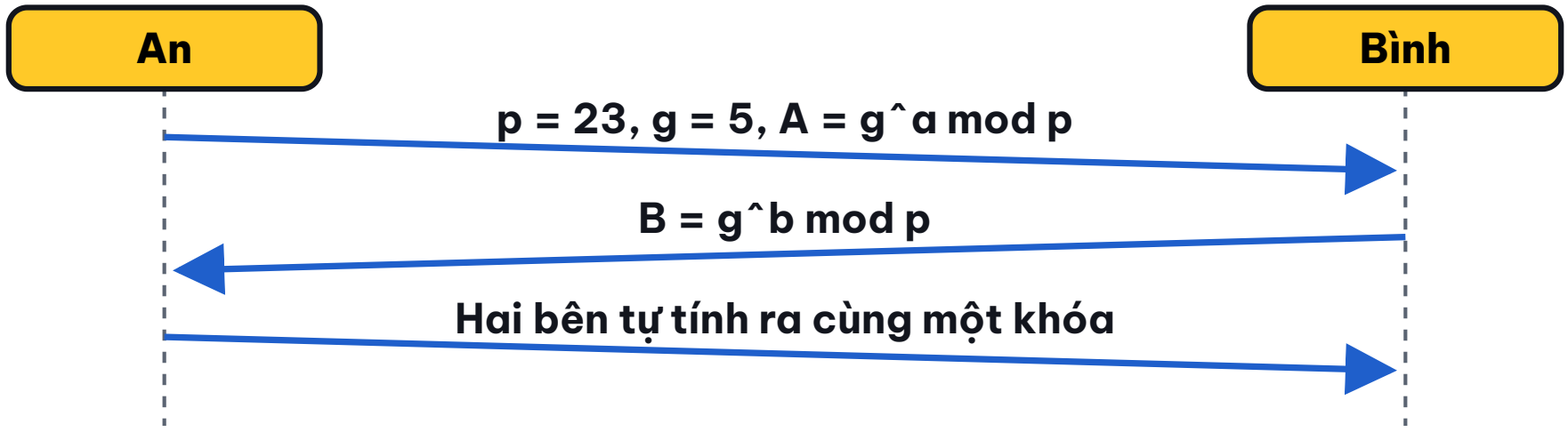
Tối thiểu 2048 bit cho hệ thống mới.

ĐỪNG DÙNG RSA THÔ

Thực tế phải dùng đệm OAEP. Không tự cài đặt, hãy dùng thư viện đã kiểm chứng.

15 Trao đổi khóa Diffie-Hellman

Hai bên cùng tạo ra một khóa bí mật qua kênh công khai, mà khóa đó không hề được gửi đi.




```python
p, g = 23, 5
a, b = 6, 15                          # hai số bí mật
A, B = pow(g, a, p), pow(g, b, p)
print("Gửi công khai:", A, B)
print("Khóa chung:", pow(B, a, p), pow(A, b, p))
```

KẾT QUẢ KHI CHẠY

```
Gửi công khai: 8 19
Khóa chung: 2 2
```

- An toàn dựa trên bài toán logarit rời rạc.
- Ephemeral DH (DHE, ECDHE): mỗi phiên một khóa mới, nên lộ khóa dài hạn không giải mã được phiên cũ.

ĐIỂM YẾU

Bản thân Diffie-Hellman không xác thực. Kẻ đứng giữa có thể đóng giả mỗi bên, nên cần chữ ký số hoặc chứng chỉ đi kèm.

16 ECC và mật mã hậu lượng tử

Hệ	Dựa trên	Dùng cho
RSA	Phân tích thừa số	Mã hóa, chữ ký
DH	Logarit rời rạc	Trao đổi khóa
ECC	Logarit rời rạc trên đường cong	Khóa ngắn hơn, nhanh hơn

Thuật toán Shor trên máy tính lượng tử đủ lớn có thể phá RSA, DH và ECC. Mã đối xứng và hàm băm ít bị ảnh hưởng hơn.

Chuẩn NIST 2024	Tên	Công dụng
FIPS 203	ML-KEM	Thiết lập khóa
FIPS 204	ML-DSA	Chữ ký số
FIPS 205	SLH-DSA	Chữ ký số dựa trên băm

Kẻ tấn công có thể thu thập dữ liệu mã hóa hôm nay để giải mã sau này, nên dữ liệu cần giữ kín lâu năm nên chuyển đổi sớm.

17 Hàm băm

Biến dữ liệu bất kỳ thành chuỗi độ dài cố định. Băm không phải mã hóa, vì không có khóa và không giải ngược được.

```python
import hashlib

a = hashlib.sha256(b"Diem: 9").digest()
b = hashlib.sha256(b"Diem: 8").digest()
print(a.hex()[:32])
print(b.hex()[:32])

diff = sum(bin(x ^ y).count("1") for x, y in zip(a, b))
print(diff, "trên 256 bit khác nhau")
```

KẾT QUẢ KHI CHẠY

```
5136b1aa9a82feda7f69af546fe8cfe0
8db276021ae282deea34ac914ee33dec
103 trên 256 bit khác nhau
```

Tính chất	Ý nghĩa
Một chiều	Từ giá trị băm khó tìm lại dữ liệu
Kháng tiền ảnh thứ hai	Cho x, khó tìm y khác có cùng giá trị băm
Kháng va chạm	Khó tìm hai dữ liệu bất kỳ cùng giá trị băm
Hiệu ứng tuyết lở	Đổi 1 bit làm đổi khoảng một nửa số bit

Hàm	Đầu ra	Tình trạng
MD5	128 bit	Đã có va chạm, không dùng cho an toàn
SHA-1	160 bit	Đã có va chạm, không dùng
SHA-256, SHA-3	256 bit trở lên	Nên dùng

18 Nghịch lý ngày sinh

Tìm va chạm dễ hơn nhiều so với cảm giác: hàm băm n bit chỉ cần khoảng 2^(n/2) lần thử.

```
from math import exp

def collision(people, days=365):
    p = 1.0
    for i in range(people):
        p *= (days - i) / days
    return 1 - p

for k in (10, 23, 50, 70):
    print(k, "người:", round(collision(k) * 100, 1), "%")

print("Băm 64 bit: khoảng 2 mũ", 64 // 2, "lần thử")
print("SHA-256: khoảng 2 mũ", 256 // 2, "lần thử")
```

KẾT QUẢ KHI CHẠY

```
10 người: 11.7 %
23 người: 50.7 %
50 người: 97.0 %
70 người: 99.9 %
Băm 64 bit: khoảng 2 mũ 32 lần thử
SHA-256: khoảng 2 mũ 128 lần thử
```

- ✓ Chỉ cần 23 người là xác suất trùng ngày sinh đã vượt 50%.
- ✓ Vì vậy độ an toàn trước va chạm chỉ bằng một nửa độ dài đầu ra.
- ✓ Đây là lý do MD5 128 bit không đủ, ngoài việc đã có tấn công cụ thể.

19 MAC và HMAC

Băm không có khóa nên ai cũng tính lại được. MAC dùng khóa bí mật để chứng minh thông điệp không bị sửa và đến từ người có khóa.

```python
import hashlib, hmac

key = b"khoa-bi-mat-chung"
msg = b"chuyen 100000 cho An"
tag = hmac.new(key, msg, hashlib.sha256).digest()

def verify(key, msg, tag):
    want = hmac.new(key, msg, hashlib.sha256).digest()
    return hmac.compare_digest(want, tag)

print(verify(key, msg, tag))
print(verify(key, b"chuyen 900000 cho An", tag))
```

KẾT QUẢ KHI CHẠY

```
True
False
```

Cơ chế	Toàn vẹn	Xác thực	Chống chối bỏ
Hàm băm	Một phần	Không	Không
MAC	Có	Có	Không
Chữ ký số	Có	Có	Có

- ✓ Dùng `compare_digest` để so sánh trong thời gian cố định.
- ✓ Không tự ghép SHA-256(khóa + dữ liệu), vì có tấn công mở rộng độ dài. Hãy dùng HMAC.

20 Chữ ký số

Ký bằng khóa bí mật, ai có khóa công khai cũng kiểm tra được. Vì chỉ người ký giữ khóa nên có tính chống chối bỏ.

```python
import hashlib

p, q, e = 61, 53, 17                  # ví dụ đồ chơi
n = p * q
d = pow(e, -1, (p - 1) * (q - 1))

def digest(msg):
    return int.from_bytes(hashlib.sha256(msg).digest(), "big") % n

def sign(msg):
    return pow(digest(msg), d, n)   # dùng khóa bí mật

def check(msg, s):
    return pow(s, e, n) == digest(msg)

s = sign(b"Hop dong so 7")
print(check(b"Hop dong so 7", s), check(b"Hop dong so 8", s))
```

KẾT QUẢ KHI CHẠY

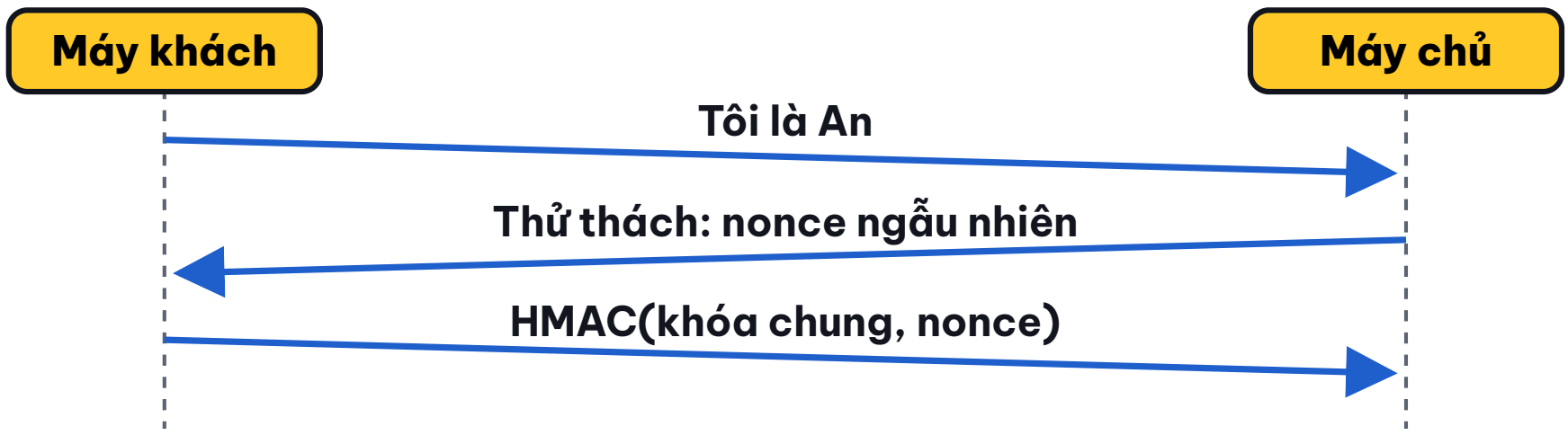
```
True False
```

Việc	Dùng khóa nào
Mã hóa cho An	Khóa công khai của An
An giải mã	Khóa bí mật của An
An ký	Khóa bí mật của An
Kiểm tra chữ ký của An	Khóa công khai của An

Thực tế dùng RSA-PSS, ECDSA, Ed25519. Ví dụ trên chỉ để minh họa.

21 Giao thức mật mã

Thuật toán tốt chưa đủ, các bước trao đổi cũng phải đúng. Thử thách và đáp ứng giúp chống gửi lại thông điệp cũ.




```python
import hashlib, hmac, secrets

key = b"khoa-chung"
def answer(nonce):
    return hmac.new(key, nonce, hashlib.sha256).digest()
nonce = secrets.token_bytes(16)
reply = answer(nonce)
print(hmac.compare_digest(reply, answer(nonce)))

new_nonce = secrets.token_bytes(16)      # lần sau: thử thách mới
print(hmac.compare_digest(reply, answer(new_nonce)))
```

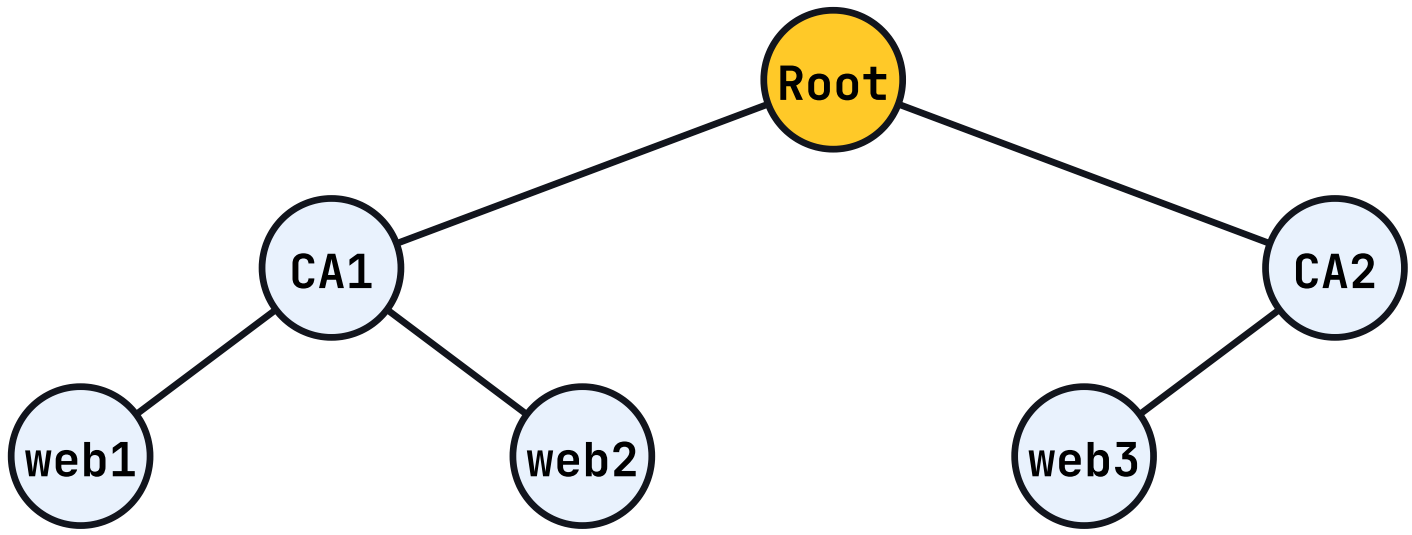
KẾT QUẢ KHI CHẠY

```
True
False
```

Khái niệm	Ý nghĩa
Nonce	Số chỉ dùng một lần, chống phát lại
Kerberos	Máy chủ trung tâm (KDC) cấp vé để đăng nhập một lần
Needham-Schroeder	Giao thức kinh điển, từng bị phát hiện lỗi sau nhiều năm

22 PKI và chứng chỉ số

Làm sao biết khóa công khai đúng là của ngân hàng? Một bên tin cậy ký xác nhận vào chứng chỉ.

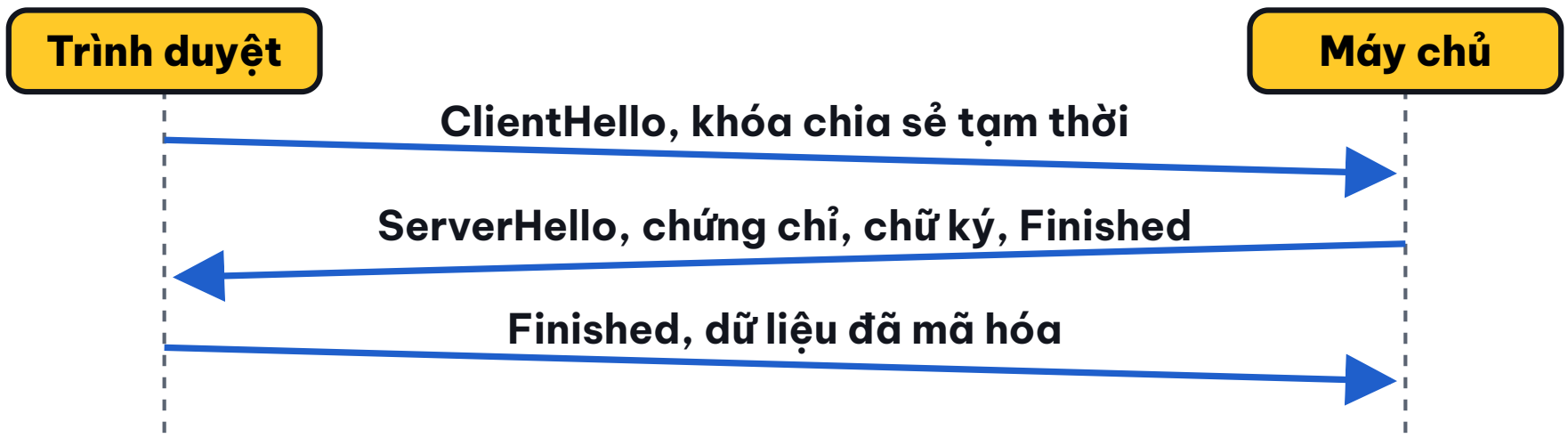


Thành phần	Vai trò
Root CA	Gốc tin cậy, có sẵn trong trình duyệt, hệ điều hành
CA trung gian	Ký chứng chỉ cho website
Chứng chỉ X.509	Gắn tên miền với khóa công khai, có hạn dùng
Thu hồi	Danh sách CRL, giao thức OCSP
Certificate Transparency	Nhật ký công khai để phát hiện chứng chỉ giả

Trình duyệt kiểm tra: chuỗi ký tới Root đáng tin, đúng tên miền, còn hạn, chưa bị thu hồi.

23 TLS bảo vệ gì?

TLS 1.3 kết hợp mọi thứ phía trên: trao khóa, chứng chỉ, mã hóa có xác thực.



Bảo vệ	Không bảo vệ
Nội dung trên đường truyền	Dữ liệu sau khi đã tới máy chủ
Chống sửa đổi dọc đường	Website lừa đảo cũng có thể có HTTPS
Xác thực tên miền máy chủ	Máy tính đã bị nhiễm mã độc

- ✓ Dùng TLS 1.2 hoặc 1.3. TLS 1.0 và 1.1 đã bị loại bỏ.
- ✓ Không bỏ qua cảnh báo chứng chỉ, nhất là trên Wi-Fi công cộng.
- ✓ HSTS buộc trình duyệt luôn dùng HTTPS với website đó.

Sai lầm mật mã hay gặp

Sai lầm	Cách đúng
Tự nghĩ ra thuật toán	Dùng chuẩn và thư viện đã kiểm chứng
Base64 coi như mã hóa	Base64 chỉ là mã hóa ký tự, ai cũng đọc được
Lưu khóa trong mã nguồn	Dùng biến môi trường, kho bí mật
Dùng random thường cho khóa	Dùng `secrets`, CSPRNG
Lặp lại IV, nonce	Sinh mới theo đúng yêu cầu của chế độ
Mã hóa không xác thực	Dùng AEAD như AES-GCM
Băm mật khẩu bằng SHA-256	Dùng Argon2id, scrypt, bcrypt

Trong đồ án, hãy chọn thư viện có sẵn. Rất hiếm khi bạn cần tự cài đặt thuật toán mật mã.

24 Xác thực là gì?

Xác thực trả lời câu hỏi "bạn là ai?". Phân quyền trả lời "bạn được làm gì?". Hai việc này khác nhau.

Yếu tố	Ý nghĩa	Ví dụ
Thứ bạn biết	Bí mật trong đầu	Mật khẩu, mã PIN
Thứ bạn có	Vật giữ trong tay	Điện thoại, khóa bảo mật
Thứ bạn là	Đặc điểm sinh trắc	Vân tay, khuôn mặt

MFA
Dùng từ hai yếu tố khác loại. Mật khẩu cộng câu hỏi bí mật vẫn chỉ là một loại.

SINH TRẮC
Không đổi được khi lộ, và có tỉ lệ nhận nhầm, từ chối nhầm.

OTP qua SMS tốt hơn không có gì, nhưng có thể bị lừa lấy mã hoặc chiếm SIM. Ứng dụng tạo mã và passkey an toàn hơn.

25 Lưu mật khẩu đúng cách

Lời giải câu đố ở bìa: băm nhanh thì dễ thử hàng loạt, và cùng mật khẩu cho cùng kết quả. Cần thêm salt và hàm băm chậm.

```python
import hashlib, hmac, os

def derive(password, salt):
    return hashlib.scrypt(password.encode(), salt=salt,
                          n=2 ** 14, r=8, p=1)

def store(password):
    salt = os.urandom(16)
    return salt, derive(password, salt)

def check(password, salt, stored):
    return hmac.compare_digest(derive(password, salt), stored)

s1, h1 = store("123456")
s2, h2 = store("123456")
print(h1 == h2)                    # cùng mật khẩu, khác kết quả
print(check("123456", s1, h1), check("654321", s1, h1))
```

KẾT QUẢ KHI CHẠY

```
False
True False
```

Cách lưu	Đánh giá
Văn bản thường	Không bao giờ được làm
SHA-256 trực tiếp	Quá nhanh, trùng nhau lộ ngay
Băm có salt	Chặn bảng tính sẵn, nhưng vẫn nhanh
Argon2id, scrypt, bcrypt	Chậm có chủ đích, nên dùng

Salt không cần giữ bí mật, nhưng phải ngẫu nhiên và riêng cho mỗi người dùng.

26 Độ mạnh của mật khẩu

Độ dài quan trọng hơn ký tự lạ, với mật khẩu thật sự ngẫu nhiên.

```python
from math import log2

def bits(alphabet, length):
    return round(length * log2(alphabet), 1)

print("8 ký tự đủ loại:  ", bits(94, 8))
print("12 ký tự đủ loại: ", bits(94, 12))
print("15 chữ thường:    ", bits(26, 15))
print("6 từ ngẫu nhiên:  ", bits(7776, 6))
```

KẾT QUẢ KHI CHẠY

```
8 ký tự đủ loại:   52.4
12 ký tự đủ loại:  78.7
15 chữ thường:     70.5
6 từ ngẫu nhiên:   77.5
```

Theo NIST SP 800-63B-4 (2025)	Quy định
Dùng một mình	Tối thiểu 15 ký tự
Dùng trong MFA	Tối thiểu 8 ký tự
Độ dài tối đa	Nên cho phép ít nhất 64 ký tự
Bắt buộc đủ loại ký tự	Không được áp đặt
Đổi định kỳ	Không bắt buộc, chỉ đổi khi nghi lộ
Danh sách mật khẩu yếu	Phải kiểm tra
Trình quản lý mật khẩu	Phải cho phép

27 Mã OTP hoạt động thế nào?

Điện thoại và máy chủ dùng chung một bí mật. Mã được tính từ bí mật và thời gian, nên không cần mạng.

```
import hashlib, hmac, struct

def hotp(secret, counter, digits=6):
    mac = hmac.new(secret, struct.pack(">Q", counter),
                   hashlib.sha1).digest()
    off = mac[-1] & 15
    code = struct.unpack(">I", mac[off:off + 4])[0] & 0x7FFFFFFF
    return str(code % 10 ** digits).zfill(digits)

def totp(secret, unix_time, step=30, digits=6):
    return hotp(secret, unix_time // step, digits)

key = b"12345678901234567890"
print(hotp(key, 0))                 # khớp ví dụ trong RFC 4226
print(totp(key, 59, digits=8))   # khớp ví dụ trong RFC 6238
```

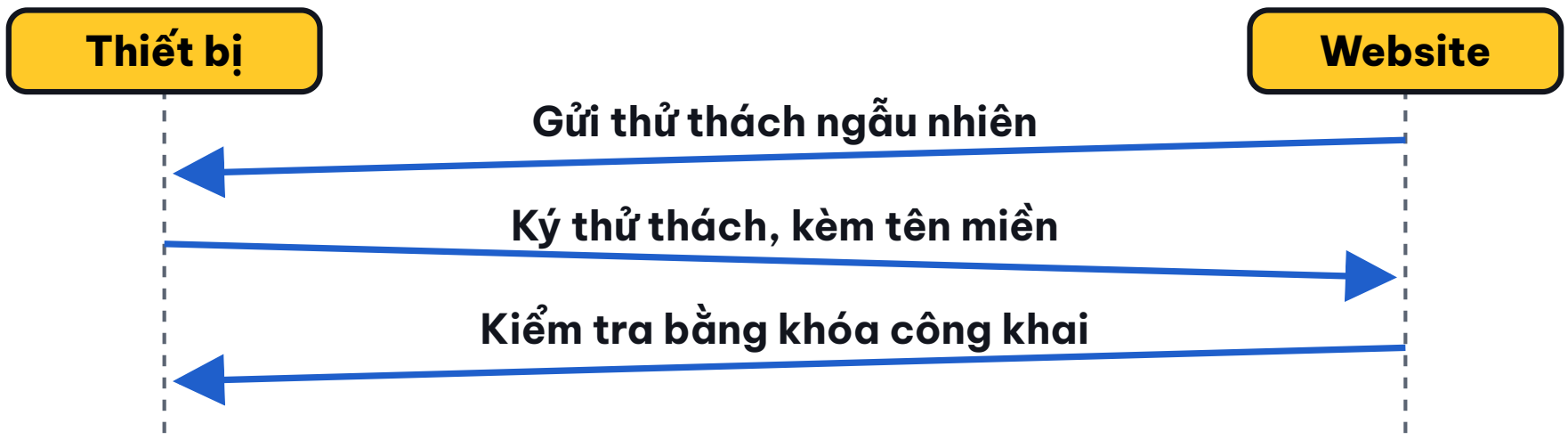
KẾT QUẢ KHI CHẠY

```
755224
94287082
```

- Mã đổi sau mỗi 30 giây, máy chủ thường chấp nhận lệch một khoảng nhỏ.
- Không ai hợp pháp cần xin mã OTP của bạn, kể cả nhân viên ngân hàng.
- OTP vẫn có thể bị lừa nhập vào trang giả, vì nó không gắn với tên miền.

28 Passkey

Passkey thay mật khẩu bằng cặp khóa. Khóa bí mật không rời thiết bị, máy chủ chỉ giữ khóa công khai.



Tiêu chí	Mật khẩu	Passkey
Lộ dữ liệu máy chủ	Lộ dữ liệu băm	Chỉ lộ khóa công khai
Trang giả	Dễ bị lừa	Gắn với tên miền nên khó lừa
Dùng lại	Hay dùng chung	Mỗi website một khóa

Khi một dịch vụ quan trọng hỗ trợ passkey, hãy bật lên và giữ sẵn cách khôi phục tài khoản.

29 Phiên đăng nhập và JWT

HTTP không nhớ gì, nên sau khi đăng nhập cần một thứ để chứng minh danh tính ở mỗi yêu cầu.

```
import base64, hashlib, hmac, json

def b64(b):
    return base64.urlsafe_b64encode(b).rstrip(b"=")

def mac(data, key):
    return b64(hmac.new(key, data, hashlib.sha256).digest())

def sign(payload, key):
    head = b64(b'{"alg":"HS256","typ":"JWT"}')
    body = b64(json.dumps(payload, separators=(",", ":")).encode())
    return head + b"." + body + b"." + mac(head + b"." + body, key)

def verify(token, key):
    head, body, sig = token.split(b".")
    return hmac.compare_digest(mac(head + b"." + body, key), sig)

key = b"khoa-rat-dai-va-ngau-nhien"
t = sign({"sub": "an", "role": "sv"}, key)
head, body, sig = t.split(b".")
forged = b64(b'{"sub":"an","role":"admin"}')
fake = head + b"." + forged + b"." + sig
print(verify(t, key), verify(fake, key))
print(base64.urlsafe_b64decode(body + b"=="))   # không được mã hóa
```

KẾT QUẢ KHI CHẠY

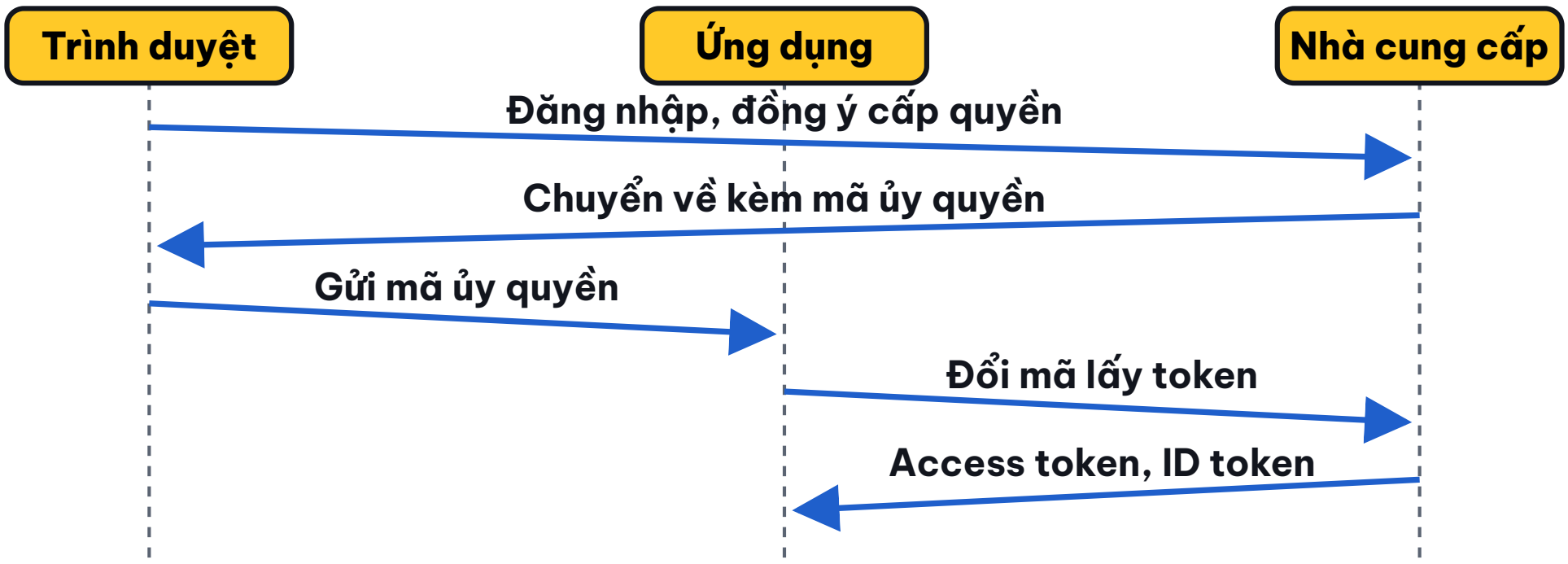
```
True False
b'{"sub":"an","role":"sv"}'
```

Cookie phiên nên bật HttpOnly (JavaScript không đọc được), Secure (chỉ gửi qua HTTPS), SameSite (hạn chế gửi từ trang khác).

Với JWT: kiểm tra chữ ký, thuật toán, hạn dùng. Không cho phép alg none.

30 OAuth 2.0 và OpenID Connect

Nút "Đăng nhập bằng Google" dựa trên OIDC. OAuth cấp quyền truy cập, OIDC bổ sung thông tin danh tính.



Thuật ngữ	Ý nghĩa
Access token	Dùng để gọi API
ID token	Cho biết người dùng là ai
Scope	Phạm vi quyền được cấp
PKCE	Chống lấy trộm mã ủy quyền
state	Chống giả mạo yêu cầu đăng nhập

Trước khi bấm "Cho phép", hãy đọc ứng dụng xin quyền gì. Thu hồi quyền của ứng dụng không còn dùng.

31 Các mô hình phân quyền

Mô hình	Ai quyết định	Ví dụ
DAC	Chủ sở hữu dữ liệu	Chia sẻ file Google Drive
MAC	Chính sách trung tâm theo mức mật	Hệ thống quân sự
RBAC	Vai trò của người dùng	Sinh viên, giảng viên, phòng đào tạo
ABAC	Thuộc tính và ngữ cảnh	Chỉ trong giờ làm, đúng khoa

```python
ROLES = {"sv": {"xem_diem"},
         "gv": {"xem_diem", "nhap_diem"},
         "daotao": {"xem_diem", "nhap_diem", "khoa_diem"}}

def can(role, action):
    return action in ROLES.get(role, set())   # lạ thì từ chối

print(can("sv", "nhap_diem"), can("gv", "nhap_diem"))
print(can("khach", "xem_diem"))
```

KẾT QUẢ KHI CHẠY

```
False True
False
```

ACL
Mỗi đối tượng có danh sách ai được làm gì.

CAPABILITY
Mỗi người giữ danh sách quyền của mình.

32 Bell-LaPadula và Biba

Hai mô hình kinh điển. Bell-LaPadula bảo vệ tính bí mật, Biba bảo vệ tính toàn vẹn.

```
rank = {"CongKhai": 0, "Mat": 1, "TuyetMat": 2}

def blp(subject, obj, action):          # bí mật
    s, o = rank[subject], rank[obj]
    return s >= o if action == "doc" else s <= o

def biba(subject, obj, action):         # toàn vẹn
    s, o = rank[subject], rank[obj]
    return s <= o if action == "doc" else s >= o

print(blp("Mat", "TuyetMat", "doc"), blp("Mat", "CongKhai", "doc"))
print(blp("Mat", "CongKhai", "ghi"), blp("Mat", "TuyetMat", "ghi"))
print(biba("Mat", "CongKhai", "doc"), biba("Mat", "CongKhai", "ghi"))
```

KẾT QUẢ KHI CHẠY

```
False True
False True
False True
```

Mô hình	Đọc	Ghi
Bell-LaPadula	Không đọc lên	Không ghi xuống
Biba	Không đọc xuống	Không ghi lên

Mẹo nhớ: Bell-LaPadula chống rò rỉ từ cao xuống thấp. Biba chống dữ liệu kém tin cậy làm bẩn dữ liệu đáng tin.

33 OWASP Top 10:2025

Danh sách nhận thức về các rủi ro quan trọng nhất của ứng dụng web.

Mã	Tên	Hiểu nhanh
A01	Broken Access Control	Làm được việc không được phép
A02	Security Misconfiguration	Cấu hình sai, mặc định yếu
A03	Software Supply Chain Failures	Thư viện, công cụ build bị xâm phạm
A04	Cryptographic Failures	Mã hóa thiếu hoặc sai
A05	Injection	Dữ liệu bị hiểu thành lệnh
A06	Insecure Design	Sai ngay từ thiết kế
A07	Authentication Failures	Lỗi đăng nhập, phiên
A08	Software or Data Integrity Failures	Tin dữ liệu, bản cập nhật chưa kiểm tra
A09	Security Logging and Alerting Failures	Không ghi log, không cảnh báo
A10	Mishandling of Exceptional Conditions	Xử lý lỗi và tình huống bất thường sai

Thứ tự theo bản 2025 của OWASP. Đề thi dùng giáo trình cũ có thể theo bản 2021.

34 Lỗi kiểm soát truy cập

Đổi số trên đường dẫn mà xem được đơn của người khác: lỗi IDOR. Luôn kiểm tra quyền phía máy chủ, trên đúng đối tượng.

```
orders = {1: {"owner": "an", "total": 100},
          2: {"owner": "binh", "total": 200}}

def bad(user, oid):
    return orders[oid]                  # không hề kiểm tra chủ

def good(user, oid):
    o = orders.get(oid)
    if o is None or o["owner"] != user:
        return "403 Forbidden"
    return o

print(bad("an", 2))
print(good("an", 2))
print(good("an", 1))
```

KẾT QUẢ KHI CHẠY

```
{'owner': 'binh', 'total': 200}
403 Forbidden
{'owner': 'an', 'total': 100}
```

- ✓ Ẩn nút trên giao diện không phải là phân quyền.
- ✓ Mặc định từ chối, chỉ mở những gì cần.
- ✓ Kiểm tra lại quyền ở mọi API, không chỉ ở trang đầu tiên.
- ✓ Viết test: người A không xem được dữ liệu của người B.

35 SQL Injection

Dữ liệu người dùng bị ghép thẳng vào câu lệnh nên biến thành lệnh. Cách sửa gốc là truy vấn tham số hóa.

```python
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE users(name TEXT, pw TEXT)")
db.execute("INSERT INTO users VALUES ('admin', 'bimat')")

def bad(name, pw):
    q = f"SELECT name FROM users WHERE name='{name}' AND pw='{pw}'"
    return db.execute(q).fetchall()

def good(name, pw):
    q = "SELECT name FROM users WHERE name=? AND pw=?"
    return db.execute(q, (name, pw)).fetchall()

evil = "x' OR '1'='1"
print(bad("admin", evil))        # vượt qua đăng nhập
print(good("admin", evil))       # chỉ là chuỗi bình thường
```

KẾT QUẢ KHI CHẠY

```
[('admin',)]
[]
```

- ✓ Dùng tham số hoặc ORM đúng cách. Đừng tự lọc dấu nháy.
- ✓ Tài khoản database chỉ có đúng quyền cần thiết.
- ✓ Không hiện lỗi SQL ra giao diện.
- ✓ Ví dụ lưu mật khẩu thường chỉ để minh họa, thực tế phải băm.

36 XSS

Nội dung người dùng được trình duyệt hiểu thành mã. Cách sửa chính là mã hóa đầu ra theo đúng ngữ cảnh.

```
import html

comment = '<script>alert("xss")</script>'

print(f"<p>{comment}</p>")                  # nguy hiểm
print(f"<p>{html.escape(comment)}</p>")     # hiển thị như chữ
```

KẾT QUẢ KHI CHẠY

```
<p><script>alert("xss")</script></p>
<p>&lt;script&gt;alert(&quot;xss&quot;)&lt;/script&gt;</p>
```

Loại	Dữ liệu độc nằm ở đâu
Stored	Lưu trong database, ai xem cũng dính
Reflected	Nằm trong đường dẫn, phản hồi ngay
DOM-based	Xử lý sai trong JavaScript phía trình duyệt

- ✓ Framework hiện đại tự escape, đừng tắt cơ chế đó khi chưa hiểu rõ.
- ✓ Content-Security-Policy là lớp phòng thủ thứ hai.
- ✓ Cookie HttpOnly giảm thiệt hại khi có XSS.
- ✓ Cần cho phép HTML thì dùng thư viện làm sạch đã kiểm chứng.

37 CSRF

Trang độc khiến trình duyệt của bạn gửi yêu cầu kèm cookie đăng nhập, mà bạn không hề hay biết.

```python
import hmac, secrets

sessions = {}

def login():
    sid = secrets.token_hex(8)
    sessions[sid] = secrets.token_urlsafe(16)   # riêng từng phiên
    return sid

def transfer(sid, token):
    want = sessions.get(sid, "")
    return bool(want) and hmac.compare_digest(want, token)

sid = login()
print(transfer(sid, sessions[sid]))    # form thật có token
print(transfer(sid, "doan-bua"))       # trang lạ không biết token
```

KẾT QUẢ KHI CHẠY

```
True
False
```

- ✓ Dùng CSRF token cho thao tác thay đổi dữ liệu.
- ✓ Đặt cookie SameSite=Lax hoặc Strict.
- ✓ Không dùng GET để thay đổi dữ liệu.
- ✓ Việc quan trọng nên yêu cầu xác thực lại.

XSS lợi dụng việc người dùng tin website. CSRF lợi dụng việc website tin trình duyệt của người dùng.

38 Path traversal và SSRF

Đừng tin đường dẫn và URL người dùng đưa vào. Hãy chuẩn hóa rồi mới kiểm tra.

```
import ipaddress, posixpath
from urllib.parse import urlparse

BASE = "/srv/files"

def safe_path(name):
    path = posixpath.normpath(posixpath.join(BASE, name))
    return path if path.startswith(BASE + "/") else None

print(safe_path("bai-tap.pdf"))
print(safe_path("../../etc/passwd"))

ALLOWED = {"api.example.com"}

def safe_url(url):
    u = urlparse(url)
    return u.scheme == "https" and u.hostname in ALLOWED

print(safe_url("https://api.example.com/v1"))
print(safe_url("http://169.254.169.254/latest"))
print(ipaddress.ip_address("169.254.169.254").is_link_local)
```

KẾT QUẢ KHI CHẠY

```
/srv/files/bai-tap.pdf
None
True
False
True
```

PATH TRAVERSAL

Dùng ../ để đọc file ngoài thư mục cho phép.

SSRF

Lừa máy chủ gọi tới địa chỉ nội bộ, như dịch vụ metadata của đám mây.

39 Command injection và tải file

```python
import re

def bad(host):
    return "ping -c 1 " + host              # chuỗi cho shell

def good(host):
    if not re.fullmatch(r"[a-z0-9.-]{1,253}", host):
        raise ValueError("host không hợp lệ")
    return ["ping", "-c", "1", host]        # danh sách đối số

print(bad("a.vn; whoami"))
print(good("a.vn"))

MAGIC = {b"\x89PNG": "png", b"\xff\xd8\xff": "jpg", b"%PDF": "pdf"}


def allow(name, data, limit=5_000_000):
    ext = name.rsplit(".", 1)[-1].lower()
    real = next((MAGIC[k] for k in MAGIC if data.startswith(k)), None)
    return len(data) <= limit and real == ext

print(allow("anh.png", b"\x89PNG\r\n"), allow("anh.png", b"<?php"))
```

KẾT QUẢ KHI CHẠY

```
ping -c 1 a.vn; whoami
['ping', '-c', '1', 'a.vn']
True False
```

- Không ghép chuỗi vào shell. Gọi hàm thư viện hoặc truyền danh sách đối số.
- Tải file: kiểm tra đuôi, nội dung thật, dung lượng.
- Đặt tên file mới và lưu ngoài thư mục chạy code.
- Không nạp dữ liệu không tin cậy bằng `pickle`.

40 Cấu hình an toàn

HTTP header	Tác dụng
Content-Security-Policy	Giới hạn nguồn script và tài nguyên
Strict-Transport-Security	Buộc dùng HTTPS
X-Content-Type-Options	Không đoán kiểu nội dung
Referrer-Policy	Giảm lộ đường dẫn khi chuyển trang
frame-ancestors	Chống nhúng trang để lừa bấm

Lỗi thường gặp	Cách sửa
Mật khẩu mặc định	Đổi ngay khi cài đặt
Bật chế độ debug trên server	Tắt ở môi trường thật
Lộ file .env, .git, backup	Chặn truy cập, không đưa lên web
Kho lưu trữ đám mây công khai	Mặc định riêng tư
Mở cổng database ra Internet	Chỉ cho phép mạng nội bộ
CORS cho phép mọi nguồn	Chỉ cho các nguồn cần thiết

Đẩy nhầm khóa bí mật lên GitHub thì phải thu hồi khóa ngay, xóa khỏi lịch sử là chưa đủ.

41 Chuỗi cung ứng phần mềm

Phần lớn code trong dự án là của người khác. Một thư viện bị xâm phạm có thể ảnh hưởng rất nhiều sản phẩm.

```
import hashlib

package = b"noi dung goi phan mem"
pinned = hashlib.sha256(package).hexdigest()

def verify(data):
    return hashlib.sha256(data).hexdigest() == pinned

print(verify(package), verify(package + b" da bi sua"))
```

KẾT QUẢ KHI CHẠY

```
True False
```

Rủi ro	Biện pháp
Gói giả tên gần giống	Kiểm tra đúng tên, đúng tác giả
Thư viện có lỗ hổng	Quét phụ thuộc, cập nhật đều
Phiên bản bị thay đổi	Khóa phiên bản bằng lockfile
Pipeline bị chiếm	Giới hạn quyền token, bật MFA
Không biết mình dùng gì	Lập SBOM

42 Lỗi, log và cảnh báo

Khi gặp lỗi, hệ thống phải về trạng thái an toàn. Log phải đủ để điều tra, nhưng không chứa bí mật.

```
def lookup(user):
    raise TimeoutError("mất kết nối dịch vụ phân quyền")

def can_open(user):
    try:
        return lookup(user)
    except Exception:
        return False                # lỗi thì từ chối

print(can_open("an"))

def mask(text):
    return text[:2] + "*" * (len(text) - 4) + text[-2:]

print(mask("0912345678"), mask("an@example.com"))
```

KẾT QUẢ KHI CHẠY

```
False
09******78 an**********om
```

NÊN GHI

Đăng nhập, đổi quyền, lỗi xác thực, thao tác nhạy cảm, thời gian, người thực hiện.

KHÔNG GHI

Mật khẩu, token, số thẻ, dữ liệu cá nhân chưa che.

- ✓ Thông báo lỗi cho người dùng phải ngắn gọn, chi tiết chỉ để trong log.
- ✓ Log nên gửi về nơi tập trung để kẻ xâm nhập khó xóa.
- ✓ Log không ai đọc thì không có giá trị, hãy đặt cảnh báo.

43 Bảo vệ API

Giới hạn tần suất giúp giảm dò mật khẩu và lạm dụng. Token bucket là thuật toán rất phổ biến.

```
class Bucket:
    def __init__(self, rate, capacity):
        self.rate, self.cap = rate, capacity
        self.tokens, self.last = capacity, 0.0

    def allow(self, now):
        passed = now - self.last
        self.tokens = min(self.cap, self.tokens + passed * self.rate)
        self.last = now
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False

b = Bucket(rate=1, capacity=3)
print([b.allow(t) for t in (0, 0, 0, 0, 1, 1, 5)])
```

KẾT QUẢ KHI CHẠY

```
[True, True, True, False, True, False, True]
```

Việc cần làm	Lý do
Kiểm tra quyền từng đối tượng	Chống xem dữ liệu của người khác
Chỉ trả trường cần thiết	Tránh lộ dữ liệu thừa
Kiểm tra kích thước, kiểu dữ liệu	Chống dữ liệu độc, quá tải
Quản lý phiên bản, tắt API cũ	Giảm bề mặt tấn công

44 An toàn trong vòng đời phần mềm

Giai đoạn	Hoạt động
Yêu cầu	Xác định dữ liệu nhạy cảm, luật áp dụng
Thiết kế	Mô hình hóa mối đe dọa
Lập trình	Code review, quy tắc code an toàn
Kiểm thử	SAST, DAST, quét phụ thuộc
Triển khai	Cấu hình an toàn, quản lý bí mật
Vận hành	Giám sát, vá lỗi, ứng cứu sự cố

SAST
Đọc mã nguồn để tìm lỗi.

DAST
Thử ứng dụng đang chạy.

SCA
Kiểm tra thư viện bên thứ ba.

SECRET SCAN
Tìm khóa lọt vào mã nguồn.

Sửa lỗi càng sớm càng rẻ. Đó là ý nghĩa của "shift left".

45 An toàn khi dùng AI

Ứng dụng dùng mô hình ngôn ngữ có thêm những rủi ro mới. Với mô hình, chỉ dẫn và dữ liệu đều là văn bản, nên dễ lẫn vào nhau.

Rủi ro	Biện pháp
Prompt injection	Coi nội dung ngoài là dữ liệu, giới hạn quyền của công cụ
Lộ dữ liệu	Không dán dữ liệu nhạy cảm vào công cụ chưa được phép
Quyền quá rộng	Đặc quyền tối thiểu, xác nhận thao tác quan trọng
Code do AI viết có lỗi	Review và test như code của người
Gói phần mềm không có thật	Kiểm tra gói trước khi cài

Coi đầu ra của mô hình là dữ liệu chưa tin cậy: kiểm tra trước khi đưa vào shell, SQL hoặc trang web.

Tham khảo thêm OWASP Top 10 for LLM Applications.

46 Tràn bộ đệm

Ghi dữ liệu vượt quá vùng nhớ sẽ đè lên dữ liệu kế bên. Đây là gốc của nhiều lỗ hổng trong C và C++.

```
mem = bytearray(5)              # 4 byte buffer, 1 byte is_admin

def unsafe(data):
    mem[:len(data)] = data              # không kiểm tra độ dài

def safe(data):
    mem[:4] = data[:4].ljust(4, b"\0") # chỉ ghi trong buffer

safe(b"ABCDE")
print(bytes(mem), "is_admin =", mem[4])
unsafe(b"ABCD\x01")
print(bytes(mem), "is_admin =", mem[4])
```

KẾT QUẢ KHI CHẠY

```
b'ABCD\x00' is_admin = 0
b'ABCD\x01' is_admin = 1
```

Biện pháp	Cách giúp
Kiểm tra biên, hàm an toàn	Ngăn ghi vượt ngay từ code
Ngôn ngữ an toàn bộ nhớ	Rust, Java, Python, Go giảm cả lớp lỗi
Stack canary	Phát hiện stack bị ghi đè
DEP / NX	Không cho chạy code trên vùng dữ liệu
ASLR	Ngẫu nhiên hóa địa chỉ bộ nhớ

Đoạn code chỉ mô phỏng ý tưởng. Python không có lỗi tràn bộ đệm kiểu C.

47 Quyền và gia cố hệ điều hành

Hệ điều hành là tuyến phòng thủ nền tảng. Sai quyền file hoặc chạy mọi thứ bằng quyền cao là lỗi rất phổ biến.

```
import stat

for mode in (0o100644, 0o100600, 0o100777, 0o040755):
    print(oct(mode)[-3:], stat.filemode(mode))
```

KẾT QUẢ KHI CHẠY

```
644 -rw-r--r--
600 -rw-------
777 -rwxrwxrwx
755 drwxr-xr-x
```

Số	Quyền
4, 2, 1	Đọc, ghi, thực thi
644	Chủ đọc ghi, người khác chỉ đọc
600	Chỉ chủ đọc ghi, hợp với file khóa
777	Ai cũng toàn quyền, tránh dùng

- ✓ Dùng tài khoản thường hằng ngày, chỉ nâng quyền khi cần.
- ✓ Tắt dịch vụ, cổng không dùng, gỡ phần mềm thừa.
- ✓ Bật cập nhật, tường lửa và mã hóa ổ đĩa.
- ✓ Tham khảo bộ khuyến nghị CIS Benchmarks.

48 An toàn cho tiến trình

Nếu một chương trình bị chiếm, thiệt hại phải được giới hạn. Đó là mục tiêu của cô lập tiến trình.

Cơ chế	Tác dụng
Kernel mode, user mode	Chương trình thường không chạm trực tiếp phần cứng
Không gian nhớ riêng	Tiến trình không đọc được bộ nhớ của nhau
Sandbox	Giới hạn việc một chương trình được làm
Container	Cô lập ở mức hệ điều hành, dùng chung kernel
Máy ảo	Cô lập mạnh hơn, có kernel riêng
SELinux, AppArmor	Kiểm soát truy cập bắt buộc trên Linux

VÍ DỤ TRÌNH DUYỆT
Mỗi trang chạy trong tiến trình riêng có quyền thấp, nên lỗi ở một tab khó ảnh hưởng toàn máy.

TÁCH ĐẶC QUYỀN
Web server chỉ cần quyền cao lúc mở cổng, sau đó chạy bằng tài khoản thường.

Không chạy ứng dụng web bằng quyền root, và đừng chạy container với quyền đặc biệt khi không cần.

49 Phần mềm độc hại

Loại	Đặc điểm
Virus	Bám vào file, cần người chạy
Worm	Tự lây qua mạng
Trojan	Giả làm phần mềm hữu ích
Ransomware	Mã hóa dữ liệu rồi đòi tiền
Spyware, Keylogger	Theo dõi, ghi lại phím gõ
Rootkit	Ẩn mình ở tầng sâu của hệ thống
Botnet	Mạng máy bị điều khiển từ xa
Infostealer	Lấy mật khẩu, cookie đã lưu

Nguồn lây hay gặp với sinh viên: phần mềm bẻ khóa, file đính kèm, tiện ích trình duyệt lạ, USB lạ.

Dấu hiệu: máy chậm bất thường, tài khoản đăng nhập lạ, phần mềm tự cài, file đổi đuôi, trình duyệt tự chuyển trang.

50 Ransomware và sao lưu

Sao lưu tốt là biện pháp mạnh nhất. Sao lưu chưa từng khôi phục thử thì chưa thể tin.

```
backup_every = 6              # giờ giữa hai lần sao lưu
restore_hours = 3             # thời gian khôi phục dự kiến
rpo, rto = 4, 8               # yêu cầu của doanh nghiệp

print("Mất tối đa", backup_every, "giờ dữ liệu")
print("Đạt RPO:", backup_every <= rpo)
print("Đạt RTO:", restore_hours <= rto)
```

KẾT QUẢ KHI CHẠY

```
Mất tối đa 6 giờ dữ liệu
Đạt RPO: False
Đạt RTO: True
```

Khái niệm	Ý nghĩa
RPO	Chấp nhận mất dữ liệu trong bao lâu
RTO	Phải khôi phục dịch vụ trong bao lâu
Quy tắc 3-2-1	3 bản, 2 loại lưu trữ, 1 bản ở nơi khác
Bản bất biến, ngoại tuyến	Kẻ tấn công khó xóa hoặc mã hóa

KHI BỊ NHIỄM
Ngắt mạng máy đó, báo ngay người phụ trách, giữ lại bằng chứng, khôi phục từ bản sao lưu sạch. Trả tiền không đảm bảo lấy lại được dữ liệu.

51 Quản lý lỗ hổng và CVSS

CVE là mã lỗ hổng. CVSS chấm mức nghiêm trọng từ 0 đến 10.

```python
import math

AV = {"N": .85, "A": .62, "L": .55, "P": .2}
AC = {"L": .77, "H": .44}
UI = {"N": .85, "R": .62}
CIA = {"H": .56, "L": .22, "N": 0}
PR = {"N": (.85, .85), "L": (.62, .68), "H": (.27, .5)}

def roundup(x):
    return math.ceil(round(x * 100000) / 10000) / 10

def cvss31(av, ac, pr, ui, s, c, i, a):
    changed = s == "C"
    iss = 1 - (1 - CIA[c]) * (1 - CIA[i]) * (1 - CIA[a])
    impact = (7.52 * (iss - .029) - 3.25 * (iss - .02) ** 15
              if changed else 6.42 * iss)
    expl = 8.22 * AV[av] * AC[ac] * PR[pr][changed] * UI[ui]
    if impact <= 0:
        return 0.0
    total = impact + expl
    return roundup(min(1.08 * total if changed else total, 10))

print(cvss31("N", "L", "N", "N", "U", "H", "H", "H"))
print(cvss31("N", "L", "N", "R", "C", "L", "L", "N"))
print(cvss31("L", "L", "L", "N", "U", "H", "H", "H"))
```

KẾT QUẢ KHI CHẠY

```
9.8
6.1
7.8
```

Mức: Thấp từ 0,1, Trung bình từ 4,0, Cao từ 7,0, Nghiêm trọng từ 9,0. CVSS chưa phải toàn bộ rủi ro. Đã có bản 4.0.

52 Các tấn công mạng thường gặp

Tấn công	Vi phạm	Phòng chống
Nghe lén	Bí mật	TLS, VPN
Giả mạo ARP	Toàn vẹn	Dynamic ARP Inspection
Giả mạo DNS	Toàn vẹn	DNSSEC, DoH, DoT
Kẻ đứng giữa	Bí mật, toàn vẹn	Chứng chỉ, HSTS
Phát lại	Xác thực	Nonce, dấu thời gian
DoS, DDoS	Sẵn sàng	Lọc, CDN, giới hạn

Cẩm nang Mạng máy tính của Code4Life giải thích chi tiết ARP, DNS, TCP và TLS ở tầng giao thức.

53 Firewall, IDS và WAF

```
rules = [("allow", "tcp", 443), ("allow", "tcp", 22),
         ("deny", "any", None)]

def decide(proto, port):
    for action, p, n in rules:                # gặp dòng khớp đầu tiên
        if p in (proto, "any") and n in (port, None):
            return action

print(decide("tcp", 443), decide("tcp", 3389), decide("udp", 53))
```

KẾT QUẢ KHI CHẠY

```
allow deny deny
```

Thiết bị	Làm gì
Firewall lọc gói	Xét IP, cổng, giao thức
Firewall có trạng thái	Nhớ kết nối đang mở
WAF	Lọc lưu lượng HTTP
IDS	Phát hiện và cảnh báo
IPS	Phát hiện và chặn

DỰA TRÊN CHỮ KÝ
Khớp mẫu đã biết. Chính xác, nhưng khó phát hiện cái mới.

DỰA TRÊN BẤT THƯỜNG
So với mức bình thường. Có thể báo nhầm nhiều hơn.

DMZ là vùng đặt máy chủ công khai, tách khỏi mạng nội bộ.

54 VPN và Zero Trust

Mô hình cũ tin mọi thứ bên trong mạng. Zero Trust không tin mặc định, luôn kiểm tra người dùng, thiết bị và ngữ cảnh.

Nguyên tắc Zero Trust	Ý nghĩa
Xác minh tường minh	Kiểm tra ở mỗi lần truy cập
Đặc quyền tối thiểu	Chỉ cấp đúng và đủ thời gian
Giả định đã bị xâm nhập	Phân vùng, giám sát, giảm phạm vi ảnh hưởng

VPN GIÚP GÌ

Mã hóa từ máy bạn tới máy chủ VPN, nối từ xa vào mạng công ty.

VPN KHÔNG GIÚP GÌ

Không chống lừa đảo, không diệt mã độc. Nhà cung cấp VPN vẫn thấy được lưu lượng của bạn.

Wi-Fi công cộng: tránh bỏ qua cảnh báo chứng chỉ, tắt chia sẻ file, không đăng nhập ở trang lạ.

55 DDoS

Làm quá tải hệ thống bằng rất nhiều yêu cầu. Kiểu khuếch đại dùng giả mạo IP để máy chủ khác trả dữ liệu lớn về nạn nhân.

```
def amplification(request_bytes, response_bytes):
    return response_bytes / request_bytes

factor = amplification(60, 3000)
print("Hệ số khuếch đại:", factor)
print("1 Gbps gửi đi thành", factor, "Gbps tới nạn nhân")
```

KẾT QUẢ KHI CHẠY

```
Hệ số khuếch đại: 50.0
1 Gbps gửi đi thành 50.0 Gbps tới nạn nhân
```

Tầng	Kiểu	Giảm thiểu
Băng thông	Khuếch đại UDP	Nhà mạng, dịch vụ chống DDoS
Giao thức	SYN flood	SYN cookie
Ứng dụng	Làm ngập HTTP	CDN, WAF, giới hạn tần suất

Nhà mạng lọc gói có địa chỉ nguồn giả (BCP 38) giúp giảm cả một lớp tấn công cho toàn Internet.

LƯU Ý

Thuê hoặc thử dịch vụ DDoS nhằm vào website khác là hành vi vi phạm pháp luật, kể cả "cho vui".

56 An toàn email

Giao thức email gốc không xác minh người gửi, nên địa chỉ gửi có thể bị giả mạo.

Cơ chế	Trả lời câu hỏi
SPF	Máy chủ này có được gửi thay tên miền không?
DKIM	Thư có chữ ký đúng của tên miền không?
DMARC	Nếu kiểm tra thất bại thì xử lý ra sao?

KIỂM TRA THƯ
Tên miền người gửi, đường dẫn thật, yêu cầu gấp, file đính kèm lạ.

KHI NGHI NGỜ
Liên hệ qua kênh đã biết, không dùng số trong thư.

Thư lừa đảo ngày nay có thể rất trơn tru vì được AI hỗ trợ, nên không thể chỉ dựa vào lỗi chính tả.

57 Kỹ nghệ xã hội

Tấn công vào tâm lý con người. Dễ hơn nhiều so với phá mật mã.

Đòn tâm lý	Ví dụ
Quyền lực	Giả công an, giảng viên, sếp
Khẩn cấp	Tài khoản sắp bị khóa trong 10 phút
Sợ hãi	Bạn liên quan đến vụ án
Tham lam	Việc nhẹ lương cao, trúng thưởng
Thân thiết	Tài khoản bạn bè nhắn vay tiền

Hình thức	Kênh
Phishing	Email, trang giả
Smishing	Tin nhắn
Vishing	Cuộc gọi
Deepfake	Giọng nói, video giả

Ba bước khi nghi ngờ: dừng lại, kiểm tra qua kênh khác, hỏi người tin cậy.

Lừa đảo sinh viên hay gặp

Kịch bản	Dấu hiệu
Việc nhẹ lương cao	Làm nhiệm vụ, nạp tiền rồi mới được rút
Giả công an, tòa án	Đe dọa qua điện thoại, đòi chuyển tiền
Giả ngân hàng	Gửi link, xin OTP, cài ứng dụng lạ
Học bổng, du học giả	Đóng phí trước, thúc giục gấp
Mượn tài khoản ngân hàng	Thuê tài khoản, có thể dính pháp lý
Nick bạn bè vay tiền	Không chịu gọi video, đổi số tài khoản
Đầu tư tiền ảo	Cam kết lãi cao, có người dẫn dắt

NẾU ĐÃ LỠ
Gọi ngay ngân hàng để khóa, đổi mật khẩu, giữ lại bằng chứng, báo cơ quan công an. Cẩn thận với người tự xưng lấy lại tiền hộ.

58 SOC và phát hiện bằng log

SOC theo dõi an toàn hằng ngày. SIEM gom log về một chỗ và tạo cảnh báo theo quy tắc.

```
from collections import Counter

logs = ["10:00:01 FAIL an 10.0.0.5",
        "10:00:03 FAIL an 10.0.0.5",
        "10:00:04 FAIL binh 10.0.0.5",
        "10:00:05 OK chi 10.0.0.8",
        "10:00:06 FAIL an 10.0.0.5",
        "10:00:09 OK an 10.0.0.5"]

fails = Counter(l.split()[3] for l in logs if " FAIL " in l)
print(fails)
print("Cảnh báo:", [ip for ip, n in fails.items() if n >= 3])

ok = {l.split()[3] for l in logs if " OK " in l}
print("Đăng nhập được sau khi thử sai:", set(fails) & ok)
```

KẾT QUẢ KHI CHẠY

```
Counter({'10.0.0.5': 4})
Cảnh báo: ['10.0.0.5']
Đăng nhập được sau khi thử sai: {'10.0.0.5'}
```

Khái niệm	Ý nghĩa
EDR	Giám sát và phản ứng trên máy trạm
IoC	Dấu vết như hash, IP, tên miền độc hại
Dương tính giả	Cảnh báo sai, gây mệt cho đội trực

59 Ứng cứu sự cố

Có kế hoạch trước khi sự cố xảy ra. Mô hình bốn giai đoạn của NIST rất hay được dạy.

1 **Chuẩn bị**
Quy trình, đầu mối liên hệ, công cụ, diễn tập.

2 **Phát hiện và phân tích**
Xác định có sự cố thật không, phạm vi ra sao.

3 **Ngăn chặn, loại bỏ, khôi phục**
Cô lập, xóa nguyên nhân, đưa hệ thống trở lại an toàn.

4 **Rút kinh nghiệm**
Viết báo cáo, sửa quy trình để không lặp lại.

Đừng vội tắt máy hay xóa file: bạn có thể làm mất bằng chứng. Hãy ngắt mạng và báo người phụ trách.

NIST SP 800-61 bản 3 (2025) đã gắn ứng cứu sự cố với khung CSF 2.0. Mô hình bốn giai đoạn đến từ bản trước và vẫn phổ biến trong giáo trình.

60 MITRE ATT&CK và điều tra số

ATT&CK là kho tri thức về hành vi tấn công đã quan sát được, giúp đội phòng thủ xây phát hiện theo hành vi.

Khái niệm	Ý nghĩa
Tactic	Mục tiêu từng giai đoạn
Technique	Cách đạt mục tiêu đó
TTP	Chiến thuật, kỹ thuật, quy trình

```
import hashlib

evidence = b"anh chup o dia"
before = hashlib.sha256(evidence).hexdigest()[:16]
after = hashlib.sha256(evidence).hexdigest()[:16]
print(before, before == after)
```

KẾT QUẢ KHI CHẠY

```
86f5dbfda7936cca True
```

Nguyên tắc điều tra số	Ý nghĩa
Thu thập theo độ bay hơi	RAM trước, ổ đĩa sau
Phân tích trên bản sao	Giữ nguyên bản gốc
Băm để chứng minh	Bằng chứng không bị thay đổi
Chuỗi hành trình	Ghi rõ ai giữ bằng chứng, lúc nào

61 An toàn đám mây

Nhà cung cấp lo an toàn của đám mây, còn bạn lo an toàn của thứ mình đặt trên đám mây.

Phần việc	IaaS	PaaS	SaaS
Dữ liệu, tài khoản	Bạn	Bạn	Bạn
Ứng dụng	Bạn	Bạn	Nhà cung cấp
Hệ điều hành	Bạn	Nhà cung cấp	Nhà cung cấp
Phần cứng, trung tâm dữ liệu	Nhà cung cấp	Nhà cung cấp	Nhà cung cấp

- ✓ Bật MFA cho tài khoản quản trị, không dùng tài khoản gốc hằng ngày.
- ✓ IAM theo đặc quyền tối thiểu, xóa khóa truy cập không dùng.
- ✓ Kiểm tra kho lưu trữ không bị mở công khai.
- ✓ Bật log, đặt cảnh báo chi phí để phát hiện bị lạm dụng.

Mức phân chia cụ thể khác nhau giữa các nhà cung cấp, hãy đọc tài liệu của họ.

62 Bảo vệ dữ liệu

Biết mình có dữ liệu gì, nhạy cảm tới đâu, rồi bảo vệ theo mức đó.

```python
import re

def redact(text):
    text = re.sub(r"\b0\d{9}\b", "0*********", text)
    return re.sub(r"\b\d{12}\b", "************", text)

print(redact("An, SĐT 0912345678, CCCD 001200012345"))
```

KẾT QUẢ KHI CHẠY

```
An, SĐT 0*********, CCCD ************
```

Trạng thái dữ liệu	Biện pháp
Khi lưu trữ	Mã hóa ổ đĩa, database
Khi truyền	TLS, VPN
Khi sử dụng	Phân quyền, che dữ liệu

- ✓ Thu thập tối thiểu: không cần thì đừng lưu.
- ✓ Che dữ liệu khi hiển thị, ẩn danh khi làm báo cáo.
- ✓ Có thời hạn lưu giữ và cách xóa an toàn.
- ✓ DLP giúp phát hiện dữ liệu nhạy cảm bị gửi ra ngoài.

Pháp luật Việt Nam

Hai đạo luật mới rất đáng chú ý. Hãy đọc văn bản gốc, vì trang này chỉ tóm tắt và không phải tư vấn pháp lý.

Văn bản	Điểm chính
Luật An ninh mạng 2025 116/2025/QH15	Thông qua 10/12/2025, hiệu lực 1/7/2026. Hợp nhất Luật An ninh mạng 2018 và Luật An toàn thông tin mạng 2015.
Luật Bảo vệ dữ liệu cá nhân 2025 91/2025/QH15	Hiệu lực 1/1/2026. Quy định quyền của chủ thể dữ liệu và nghĩa vụ của bên xử lý.

Quyền của chủ thể dữ liệu	Ví dụ
Được biết, đồng ý, rút lại	Biết dữ liệu dùng vào việc gì
Truy cập, sửa, yêu cầu xóa	Sửa thông tin sai về mình
Hạn chế, phản đối xử lý	Từ chối một mục đích xử lý

Với lập trình viên: thu thập có mục đích rõ ràng, hỏi ý kiến người dùng, và có quy trình báo cáo khi lộ dữ liệu.

Cập nhật theo thông tin công khai tới 11/2026. Các nghị định hướng dẫn có thể thay đổi.

Đạo đức và những việc không được làm

Có khả năng làm không có nghĩa là được phép làm. Biết kỹ thuật lại càng phải biết giới hạn.

Hành vi	Vì sao nguy hiểm
Thử đăng nhập tài khoản người khác	Xâm nhập trái phép
Quét, tấn công web của trường	Không có sự cho phép
Mua bán dữ liệu cá nhân	Vi phạm quyền riêng tư
Phát tán mã độc, thuê DDoS	Gây thiệt hại cho nhiều người
Bán, cho thuê tài khoản ngân hàng	Tiếp tay cho rửa tiền, lừa đảo

BỘ LUẬT HÌNH SỰ
Có nhóm tội về công nghệ thông tin, ví dụ Điều 286 phát tán chương trình gây hại, Điều 287 cản trở hoạt động mạng, Điều 289 xâm nhập trái phép, Điều 290 dùng mạng chiếm đoạt tài sản.

Thấy lỗ hổng trên hệ thống thật: dừng lại, không khai thác, không lấy dữ liệu, rồi báo cho đơn vị quản lý qua kênh chính thức.

63 ISO 27001, NIST CSF, CIS

Khung	Dùng để
ISO/IEC 27001:2022	Xây hệ thống quản lý an toàn thông tin, có thể chứng nhận
NIST CSF 2.0	Đánh giá và cải thiện năng lực an ninh mạng
CIS Controls	Danh sách việc ưu tiên cần làm

ISO 27001 Phụ lục A	Số
Tổ chức	37
Con người	8
Vật lý	14
Công nghệ	34
Tổng	93

6 chức năng CSF 2.0
Govern: Quản trị
Identify: Nhận diện
Protect: Bảo vệ
Detect: Phát hiện
Respond: Ứng phó
Recover: Khôi phục

ISO 27001 hoạt động theo chu trình PDCA. Không bắt buộc áp dụng cả 93 biện pháp, mà chọn dựa trên đánh giá rủi ro.

64 Kiểm thử an toàn hợp pháp

Kiểm thử xâm nhập khác tấn công ở chỗ có sự đồng ý rõ ràng, phạm vi rõ ràng và báo cáo cho chủ hệ thống.

Trước khi làm	Cần có
Ủy quyền	Văn bản của chủ hệ thống
Phạm vi	Hệ thống nào được thử, cái nào không
Thời gian, cách làm	Không gây gián đoạn dịch vụ
Xử lý dữ liệu	Bảo mật mọi thông tin thu được

BUG BOUNTY
Chỉ thử trong phạm vi chương trình công bố, tuân thủ đúng luật chơi.

CÔNG BỐ CÓ TRÁCH NHIỆM
Báo riêng cho bên quản lý, cho họ thời gian sửa rồi mới công bố.

Kiểm tra file security.txt của website để biết kênh báo lỗi chính thức, nếu có.

Dựng phòng lab an toàn

Thực hành trên máy ảo, mạng tách biệt và dữ liệu giả. Không dùng tài khoản thật, không thử lên hệ thống của người khác.

1. **Tạo máy ảo**
   Dùng VirtualBox hoặc VMware, cài Linux, tạo snapshot để quay lại khi cần.

2. **Tách mạng**
   Dùng chế độ host-only hoặc mạng nội bộ của máy ảo.

3. **Cài ứng dụng cố tình có lỗi**
   OWASP Juice Shop, DVWA được tạo ra để luyện tập trong lab.

4. **Ghi chép**
   Ghi lại lỗi, nguyên nhân và cách sửa, không chỉ cách khai thác.

Công cụ	Dùng để
Wireshark	Phân tích gói tin trên mạng của bạn
OWASP ZAP	Kiểm tra ứng dụng web của chính bạn
Semgrep, Trivy	Quét code, thư viện, container
OpenSSL, GnuPG	Làm việc với chứng chỉ, mã hóa, chữ ký
KeePassXC	Quản lý mật khẩu

Lab 1: Phát hiện file bị sửa

Lưu giá trị băm khi hệ thống còn sạch, rồi so lại để biết file nào đã bị thay đổi.

```
import hashlib, pathlib, tempfile

def manifest(folder):
    return {p.name: hashlib.sha256(p.read_bytes()).hexdigest()
            for p in folder.iterdir()}

with tempfile.TemporaryDirectory() as d:
    folder = pathlib.Path(d)
    (folder / "a.txt").write_text("diem 9")
    (folder / "b.txt").write_text("diem 8")
    old = manifest(folder)
    (folder / "a.txt").write_text("diem 10")
    new = manifest(folder)
    print([k for k in sorted(old) if old[k] != new[k]])
```

KẾT QUẢ KHI CHẠY

```
['a.txt']
```

BÀI TẬP

Mở rộng để phát hiện cả file mới và file bị xóa, lưu manifest ra file JSON.

CÂU HỎI

Kẻ tấn công sửa luôn manifest thì sao? Hãy cải tiến bằng HMAC, khóa cất ở nơi khác.

Trên Windows dùng `certutil -hashfile a.txt SHA256`, trên Linux dùng `sha256sum a.txt`.

Lab 2: Đăng nhập an toàn

Kết hợp truy vấn tham số, salt, hàm băm chậm và khóa tạm sau nhiều lần sai.

```
import hashlib, hmac, os, sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE users(name PRIMARY KEY, salt, hash, fails)")
def derive(pw, salt):
    return hashlib.scrypt(pw.encode(), salt=salt, n=2 ** 14, r=8, p=1)
def register(name, pw):
    salt = os.urandom(16)
    db.execute("INSERT INTO users VALUES (?, ?, ?, 0)",
               (name, salt, derive(pw, salt)))
def login(name, pw):
    q = "SELECT salt, hash, fails FROM users WHERE name=?"
    row = db.execute(q, (name,)).fetchone()
    if row is None or row[2] >= 3:
        return False
    ok = hmac.compare_digest(derive(pw, row[0]), row[1])
    db.execute("UPDATE users SET fails=? WHERE name=?",
               (0 if ok else row[2] + 1, name))
    return ok

register("an", "mat-khau-dai-de-nho")
print(login("an", "mat-khau-dai-de-nho"))
print([login("an", "sai") for _ in range(3)])
print(login("an", "mat-khau-dai-de-nho"))     # đã bị khóa tạm
```

KẾT QUẢ KHI CHẠY

```
True
[False, False, False]
False
```

BÀI TẬP

Thêm thời gian mở khóa, thông báo lỗi chung chung, và ghi log an toàn.

Lab 3: Đọc log web

Viết quy tắc đơn giản để phát hiện dấu hiệu thăm dò trong log của website bạn quản lý.

```
import re

logs = ["GET /index.html 200",
        "GET /login?u=x' OR '1'='1 200",
        "GET /../../etc/passwd 404",
        "GET /search?q=<script> 200",
        "GET /tin-tuc 200"]

rules = {"SQLi": r"' OR |UNION SELECT",
         "Traversal": r"\.\./",
         "XSS": r"<script"}

for line in logs:
    hits = [n for n, r in rules.items() if re.search(r, line, re.I)]
    if hits:
        print(hits, line)
```

KẾT QUẢ KHI CHẠY

```
['SQLi'] GET /login?u=x' OR '1'='1 200
['Traversal'] GET /../../etc/passwd 404
['XSS'] GET /search?q=<script> 200
```

- ✓ Quy tắc kiểu này dễ bị né và có thể báo nhầm, chỉ phù hợp để học.
- ✓ Mã 200 không có nghĩa tấn công thành công, cần điều tra tiếp.

BÀI TẬP

Đếm theo IP, đặt ngưỡng cảnh báo, xuất báo cáo các dấu hiệu theo giờ.

Lab 4: Kiểm tra HTTPS và luyện CTF

KIỂM TRA CHỨNG CHỈ

Bấm vào biểu tượng cạnh địa chỉ trên trình duyệt, xem tên miền, đơn vị cấp, hạn dùng, chuỗi chứng chỉ.

```
openssl s_client -connect example.com:443 -servername example.com
curl -I https://example.com
```

1 **Xem phiên bản TLS**

Nên thấy TLS 1.2 hoặc 1.3.

2 **Xem header bảo mật**

So với bảng ở phần Cấu hình an toàn.

CTF là cuộc thi được phép hack trong môi trường dựng sẵn, cách học rất hiệu quả và hợp pháp.

Nơi luyện	Hợp với
picoCTF	Người mới bắt đầu
OverTheWire	Làm quen Linux
TryHackMe	Học có lộ trình
Hack The Box	Đã có nền tảng

Chỉ tấn công những mục tiêu do nền tảng cung cấp. Luôn đọc luật chơi.

Viết báo cáo lỗ hổng

Tìm ra lỗi mới là một nửa việc. Viết rõ để người khác sửa được mới tạo ra giá trị.

Mục	Nội dung
Tiêu đề	Lỗi gì, ở đâu
Mức độ	Điểm CVSS và lý do
Mô tả	Nguyên nhân gốc
Tái hiện	Các bước trong môi trường được phép
Ảnh hưởng	Thiệt hại với người dùng, doanh nghiệp
Khắc phục	Cách sửa cụ thể
Kiểm tra lại	Cách xác nhận đã sửa xong

Đồ án gợi ý: chọn một ứng dụng web của chính nhóm bạn, rà soát theo OWASP Top 10, sửa lỗi, nộp báo cáo trước và sau khi sửa.

Danh sách an toàn cá nhân

- ✓ Dùng trình quản lý mật khẩu, mỗi tài khoản một mật khẩu riêng.
- ✓ Bật xác thực hai lớp cho email, ngân hàng, mạng xã hội, GitHub.
- ✓ Cập nhật hệ điều hành, trình duyệt, ứng dụng.
- ✓ Không cài phần mềm bẻ khóa, không cài ứng dụng ngoài kho chính thức.
- ✓ Không đưa OTP, mật khẩu cho bất kỳ ai.
- ✓ Kiểm tra đường dẫn trước khi đăng nhập.
- ✓ Sao lưu tài liệu quan trọng, ít nhất một bản ngoại tuyến.
- ✓ Khóa màn hình, mã hóa ổ đĩa, bật tìm thiết bị.
- ✓ Rà soát quyền ứng dụng và phiên đăng nhập lạ.
- ✓ Hạn chế đăng ảnh căn cước, thẻ sinh viên, vé máy bay.

Làm ngay 3 việc đầu tiên trong tuần này: trình quản lý mật khẩu, MFA cho email, sao lưu đồ án.

Danh sách cho website, đồ án

Nhóm	Việc cần làm
Đăng nhập	Băm mật khẩu đúng, giới hạn thử sai, có MFA
Phân quyền	Kiểm tra phía máy chủ trên từng đối tượng
Dữ liệu vào	Tham số hóa truy vấn, kiểm tra file tải lên
Dữ liệu ra	Escape theo ngữ cảnh, không lộ lỗi chi tiết
Truyền tải	HTTPS, HSTS, cookie an toàn
Bí mật	Không đưa vào Git, xoay vòng khóa
Thư viện	Quét lỗ hổng, cập nhật
Vận hành	Log, cảnh báo, sao lưu, kế hoạch sự cố

Với dự án thật, hãy đối chiếu thêm OWASP ASVS và OWASP Cheat Sheet Series.

Bài mẫu: mật mã

Đề 1. Mã hóa RSA với p = 11, q = 13, e = 7, m = 9. Tìm d và bản mã.

Lời giải. n = 143, φ(n) = 120. Tìm d để $7d \equiv 1 \bmod 120$, được d = 103. Bản mã $c = 9^7 \bmod 143$.

Đề 2. Diffie-Hellman với p = 17, g = 3, a = 4, b = 5. Tìm khóa chung.

Lời giải. $A = 3^4 \bmod 17 = 13$, $B = 3^5 \bmod 17 = 5$. Khóa chung là $5^4 \bmod 17$.

Đề 3. Hệ thống có 100 người, mỗi cặp một khóa đối xứng. Cần bao nhiêu khóa? Nếu dùng khóa công khai thì sao?

Lời giải. Đối xứng cần 100 × 99 / 2 khóa. Khóa công khai cần 100 cặp khóa.

```
p, q, e, m = 11, 13, 7, 9
n, phi = p * q, (p - 1) * (q - 1)
d = pow(e, -1, phi)
c = pow(m, e, n)
print(n, phi, d, c, pow(c, d, n))

P, g, a, b = 17, 3, 4, 5
A, B = pow(g, a, P), pow(g, b, P)
print(A, B, pow(B, a, P), pow(A, b, P))

print(100 * 99 // 2)
```

KẾT QUẢ KHI CHẠY

```
143 120 103 48 9
13 5 13 13
4950
```

Bài mẫu: rủi ro và phân quyền

Đề 1. Máy chủ trị giá 500 triệu. Sự cố làm hỏng 30%, trung bình 2 năm một lần. Tính SLE và ALE.

Lời giải. SLE = 500 × 0,3 = 150 triệu. ARO = 0,5. ALE = 75 triệu mỗi năm.

Đề 2. Theo Bell-LaPadula, người mức Mật có được đọc tài liệu Tuyệt mật và ghi vào tài liệu Công khai không?

Lời giải. Đều không. Không đọc lên và không ghi xuống.

Đề 3. Mật khẩu ngẫu nhiên 10 chữ thường có bao nhiêu bit entropy?

Lời giải. $10 \times \log_2 26$, xấp xỉ 47 bit.

Đề 4. Phân biệt mã hóa, băm và mã hóa ký tự Base64.

Lời giải. Mã hóa cần khóa để giải. Băm là một chiều. Base64 ai cũng giải được, không có tính bí mật.

```
from math import log2

sle = 500 * 0.3
print(sle, sle * 0.5)
print(round(10 * log2(26), 1))
```

KẾT QUẢ KHI CHẠY

```
150.0 75.0
47.0
```

Tự luyện

Tự làm trước, đáp án ở trang sau.

BÀI 1
Mã hóa Caesar chuỗi BAOMAT với k = 5.

BÀI 2
Mã hóa Vigenère chuỗi ANTOAN với khóa KEY.

BÀI 3
Tìm nghịch đảo của 3 theo modulo 26, và của 7 theo modulo 40.

BÀI 4
RSA với p = 7, q = 17, e = 5. Tìm d, rồi mã hóa m = 10.

BÀI 5
Diffie-Hellman với p = 23, g = 7, a = 3, b = 9. Tìm khóa chung.

BÀI 6
Tính $\varphi(77)$ và $2^{100} \bmod 7$.

BÀI 7
Hàm băm 80 bit cần khoảng bao nhiêu lần thử để có va chạm?

BÀI 8
Tài sản 2 tỷ, EF 25%, ARO 0,2. Tính ALE.

BÀI 9
Tính điểm CVSS 3.1 của AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N.

BÀI 10
Chuyển quyền 750 sang dạng rwx.

Đáp án tự luyện

Các đáp án dưới đây do code tính ra.

KẾT QUẢ KHI CHẠY

```
1. GFTRFY
2. KRRYEL
3. 9 23
4. n = 119 d = 77 c = 40
5. A = 21 B = 15 khóa = 17 17
6. phi = 60 | 2^100 mod 7 = 2
7. khoảng 2 mũ 40
8. ALE = 100,000,000
9. 6.5
10. rwxr-x---
```

Mẹo kiểm tra lại	Cách làm
Nghịch đảo	Nhân lại phải ra 1 theo modulo
RSA	Giải mã phải trả về m ban đầu
Diffie-Hellman	Hai bên phải cùng một khóa

Trắc nghiệm (1/2)

1. Sửa điểm trái phép vi phạm mục tiêu nào?
A. Bí mật B. Toàn vẹn C. Sẵn sàng D. Chống chối bỏ

2. DDoS chủ yếu nhắm vào mục tiêu nào?
A. Bí mật B. Toàn vẹn C. Xác thực D. Sẵn sàng

3. Thuật toán nào là mã hóa đối xứng?
A. AES B. RSA C. ECDSA D. SHA-256

4. Hàm băm nào không nên dùng cho mục đích an toàn?
A. SHA-256 B. SHA-3 C. MD5 D. SHA-512

5. An ký một văn bản bằng khóa nào?
A. Công khai của An B. Bí mật của An C. Công khai của Bình D. Bí mật của Bình

6. Gửi thư bí mật cho Bình thì mã hóa bằng khóa nào?
A. Công khai của Bình B. Bí mật của Bình C. Bí mật của mình D. Không cần khóa

7. Chế độ nào làm lộ khuôn mẫu dữ liệu?
A. GCM B. CTR C. CBC D. ECB

8. Salt dùng để làm gì?
A. Mã hóa mật khẩu B. Làm giá trị băm khác nhau, chống bảng tính sẵn C. Thay MFA D. Giấu tên đăng nhập

9. Nên dùng gì để lưu mật khẩu?
A. Base64 B. MD5 C. Argon2id D. AES với khóa trong code

10. Theo NIST SP 800-63B-4, mật khẩu dùng một mình tối thiểu mấy ký tự?
A. 6 B. 8 C. 12 D. 15

Trắc nghiệm (2/2)

11. Khóa bảo mật USB thuộc yếu tố nào?
A. Thứ bạn biết B. Thứ bạn có C. Thứ bạn là D. Nơi bạn ở

12. Bell-LaPadula quy định gì?
A. Không đọc lên, không ghi xuống B. Không đọc xuống, không ghi lên C. Đọc ghi tự do D. Chỉ quản trị viên được đọc

13. Biện pháp chính chống SQL Injection?
A. Ẩn nút đăng nhập B. Dùng HTTPS C. Truy vấn tham số hóa D. Đổi tên bảng

14. Thuộc tính HttpOnly của cookie giúp gì?
A. Nén cookie B. JavaScript không đọc được cookie C. Mã hóa cookie D. Chống SQLi

15. A01 của OWASP Top 10:2025 là gì?
A. Injection B. Cryptographic Failures C. Insecure Design D. Broken Access Control

16. Mã độc nào tự lây qua mạng?
A. Worm B. Trojan C. Rootkit D. Keylogger

17. Quy tắc sao lưu 3-2-1 nghĩa là gì?
A. 3 ngày, 2 tuần, 1 tháng B. 3 bản, 2 loại lưu trữ, 1 bản ở nơi khác C. 3 máy, 2 ổ, 1 người D. Sao lưu 3 lần mỗi ngày

18. AV = 100, EF = 0,5, ARO = 2 thì ALE bằng?
A. 25 B. 50 C. 100 D. 200

19. Thành phần xác nhận khóa công khai thuộc về tên miền?
A. DNS B. VPN C. Chứng chỉ do CA ký D. Firewall

20. Vô tình thấy lỗ hổng trên web của trường, nên làm gì?
A. Khai thác thử B. Đăng lên mạng C. Bán thông tin D. Dừng lại và báo qua kênh chính thức

Đáp án trắc nghiệm

Câu	Đáp án	Giải thích
1	B	Dữ liệu bị sửa trái phép
2	D	Làm dịch vụ không dùng được
3	A	RSA, ECDSA là bất đối xứng, SHA-256 là băm
4	C	MD5 đã có va chạm
5	B	Ký bằng khóa bí mật của mình
6	A	Chỉ Bình giải được
7	D	ECB
8	B	Salt ngẫu nhiên riêng cho từng người
9	C	Hàm băm mật khẩu chậm có chủ đích
10	D	15 ký tự
11	B	Vật giữ trong tay
12	A	Bảo vệ tính bí mật
13	C	Tách dữ liệu khỏi câu lệnh
14	B	Giảm thiệt hại khi có XSS
15	D	Broken Access Control
16	A	Worm
17	B	3 bản, 2 loại, 1 nơi khác
18	C	100 × 0,5 × 2 = 100
19	C	PKI
20	D	Không khai thác, báo đúng kênh

20 câu vấn đáp (1/2)

1. Mã hóa khác băm thế nào?
Mã hóa giải ngược được khi có khóa. Băm là một chiều, dùng để kiểm tra toàn vẹn.

2. Đối xứng khác bất đối xứng thế nào?
Đối xứng dùng chung một khóa, nhanh. Bất đối xứng dùng cặp khóa, giải bài toán trao khóa.

3. Vì sao TLS dùng cả hai?
Bất đối xứng để xác thực và thỏa thuận khóa, đối xứng để mã hóa dữ liệu nhanh.

4. MAC khác chữ ký số thế nào?
MAC dùng khóa chung nên không chống chối bỏ. Chữ ký số dùng khóa bí mật riêng.

5. Salt và pepper khác nhau thế nào?
Salt lưu cùng hash, riêng từng người. Pepper là bí mật dùng chung, lưu tách khỏi database.

6. Xác thực khác phân quyền thế nào?
Xác thực là bạn là ai. Phân quyền là bạn được làm gì.

7. Vì sao mật khẩu phải băm chậm?
Để việc đoán hàng loạt sau khi lộ database trở nên tốn kém.

8. Forward secrecy là gì?
Lộ khóa dài hạn vẫn không giải mã được phiên cũ, nhờ khóa tạm cho mỗi phiên.

9. XSS khác CSRF thế nào?
XSS chạy mã trong trang của nạn nhân. CSRF mượn phiên đăng nhập để gửi yêu cầu.

10. Vì sao chỉ lọc dữ liệu vào là chưa đủ?
Phải tách dữ liệu khỏi lệnh và escape theo ngữ cảnh đầu ra.

20 câu vấn đáp (2/2)

11. IDOR là gì?
Đổi mã đối tượng là truy cập được dữ liệu người khác, do thiếu kiểm tra quyền.

12. Đặc quyền tối thiểu là gì?
Chỉ cấp đúng quyền cần thiết, trong thời gian cần thiết.

13. Phòng thủ nhiều lớp là gì?
Nhiều biện pháp chồng lên nhau, một lớp hỏng vẫn còn lớp khác.

14. IDS khác IPS thế nào?
IDS phát hiện và cảnh báo. IPS đứng trên đường đi nên chặn được.

15. RPO khác RTO thế nào?
RPO là lượng dữ liệu chấp nhận mất. RTO là thời gian khôi phục dịch vụ.

16. Zero Trust là gì?
Không tin mặc định, xác minh liên tục, đặc quyền tối thiểu, giả định đã bị xâm nhập.

17. SAST khác DAST thế nào?
SAST đọc mã nguồn. DAST kiểm tra ứng dụng đang chạy.

18. Khi nghi máy nhiễm ransomware cần làm gì?
Ngắt mạng, báo người phụ trách, giữ bằng chứng, khôi phục từ bản sạch.

19. Vì sao đừng tự viết thuật toán mã hóa?
Rất dễ sai ở những chỗ tinh vi. Thuật toán chuẩn đã được cộng đồng kiểm chứng.

20. Kiểm thử xâm nhập khác tấn công thế nào?
Có ủy quyền, phạm vi rõ ràng, và báo cáo để sửa lỗi.

Lỗi hay mất điểm

Nhầm lẫn	Cách đúng
Băm là mã hóa	Băm là một chiều, không có khóa
Base64 là mã hóa	Chỉ là cách biểu diễn dữ liệu
Ký bằng khóa công khai	Ký bằng khóa bí mật
Salt phải giữ bí mật	Salt lưu cùng hash, phải ngẫu nhiên
HTTPS là web an toàn	Chỉ bảo vệ đường truyền
Xác thực và phân quyền	Hai việc khác nhau
Lỗ hổng và mối đe dọa	Điểm yếu khác với nguồn gây hại
Bell-LaPadula và Biba	Bí mật và toàn vẹn, quy tắc ngược nhau
Tính ALE quên ARO	ALE = AV × EF × ARO

Tra nhanh: nên và không nên dùng

Mục đích	Nên dùng	Tránh
Mã hóa dữ liệu	AES-GCM, ChaCha20-Poly1305	DES, RC4, ECB
Băm toàn vẹn	SHA-256, SHA-3	MD5, SHA-1
Lưu mật khẩu	Argon2id, scrypt, bcrypt	Băm nhanh, không salt
Chữ ký số	Ed25519, ECDSA, RSA-PSS	RSA dưới 2048 bit
Trao khóa	ECDHE, X25519	Trao khóa tĩnh
Truyền tải	TLS 1.2, 1.3	SSL, TLS 1.0, 1.1
Số ngẫu nhiên	CSPRNG, `secrets`	`random`

Khuyến nghị có thể thay đổi theo thời gian, hãy kiểm tra tài liệu mới của NIST và OWASP.

Kế hoạch 15 tuần

Tuần	Nội dung	Làm
1	Tổng quan, CIA, rủi ro	Bài ALE, STRIDE
2	Mật mã cổ điển	Viết Caesar, Vigenère
3	Mã đối xứng, chế độ	So sánh ECB, CBC
4	Modulo, RSA	Giải bài RSA bằng tay
5	Diffie-Hellman, ECC	Bài DH
6	Băm, MAC, chữ ký	Lab 1
7	PKI, TLS	Lab 4
8	Xác thực, mật khẩu	Lab 2, ôn giữa kỳ
9	Phân quyền	Bài Bell-LaPadula
10	OWASP, Injection, XSS	Sửa lỗi trong lab
11	Bộ nhớ, hệ điều hành, mã độc	Gia cố một máy ảo
12	An toàn mạng	Viết luật firewall
13	SOC, ứng cứu sự cố	Lab 3
14	Pháp luật, ISO, NIST	Đánh giá rủi ro đồ án
15	Ôn tập	Bài mẫu, trắc nghiệm, vấn đáp

Trình tự theo cẩm nang. Hãy chỉnh cho khớp đề cương của trường bạn.

Nghề và chứng chỉ

Vị trí	Công việc	Nên giỏi
SOC Analyst	Giám sát, xử lý cảnh báo	Log, mạng, hệ điều hành
Pentester	Kiểm thử có ủy quyền	Web, mạng, viết báo cáo
AppSec	Bảo mật cho đội phát triển	Lập trình, thiết kế
Cloud Security	An toàn hạ tầng đám mây	IAM, tự động hóa
GRC	Rủi ro, tuân thủ, chính sách	ISO, pháp luật

Mảng của CompTIA Security+ SY0-701	Tỉ trọng
Khái niệm chung	12%
Đe dọa, lỗ hổng, biện pháp	22%
Kiến trúc an toàn	18%
Vận hành an toàn	28%
Quản lý chương trình an toàn	20%

Nền tảng lập trình, mạng và Linux quan trọng hơn việc học thuộc tên công cụ. Hãy xem thông tin chứng chỉ trên trang chính thức trước khi đăng ký.

Tài liệu nên đọc

Nguồn	Vì sao nên đọc
Nguyễn Khanh Văn **Giáo trình Cơ sở An toàn thông tin**	Giáo trình tiếng Việt
Stallings **Cryptography and Network Security**	Mật mã và an toàn mạng
Bishop **Introduction to Computer Security**	Nền tảng lý thuyết
Anderson **Security Engineering**	Tư duy hệ thống
OWASP Cheat Sheet Series, ASVS	Hướng dẫn thực hành cho web
NIST, MITRE ATT&CK	Tiêu chuẩn và kho tri thức
Thư viện pháp luật, Công báo	Văn bản gốc của luật

Thấy chỗ nào chưa đúng? Anh em comment giúp để Code4Life sửa ở bản sau nhé.

Biên soạn: Tào Việt Đức, Code4Life Technology, với sự hỗ trợ của AI. Code Python đã chạy thử trên Python 3.14. Ví dụ mật mã dùng số nhỏ chỉ để học, không dùng cho hệ thống thật. Tài liệu này không phải tư vấn pháp lý.